

JENKINS-TASK-02

1. Configure 2 slave machines in the Jenkins master.

Configure Two Jenkins Agents with an Existing Jenkins Controller

Objective: To configure two Jenkins agent (slave) machines with a Jenkins controller (master) and execute Jenkins jobs on the agents using labels.

Note: Jenkins now officially uses the terms **Controller** and **Agent** instead of Master and Slave. For your assignment, you can mention: *“Jenkins Master (Controller) and Slave (Agent).”*

Prerequisites

Before configuring the agents, ensure the following:

1. Jenkins Controller is already installed and running.
2. Jenkins Controller is using JDK 21.
3. Two separate Linux machines/EC2 instances are available.
4. Java JDK 21 is installed on both agents.
5. Controller and agents have network connectivity.
6. SSH port 22 is allowed between the Controller and Agents.
7. Jenkins administrator access is available.

Create two EC2 Instances for Agent-01 and Agent-02

Step 1: Login to Agent 1

Connect to your first EC2 instance:

```
ssh -i your-key.pem ec2-user@<AGENT-1-IP>
```

Step 2: Check Java on Agent 1

Run:

```
java -version
```

Make sure Java 21 is installed.

If Java is not installed:

sudo dnf install java-21-amazon-corretto -y

Verify: java -version

```
root@ip-172-31-19-86:~#
Verifying : libxext-1.3.6-1.amzn2023.0.1.x86_64 23/35
Verifying : libxi-1.8.2-1.amzn2023.0.1.x86_64 24/35
Verifying : libXinerama-1.1.5-6.amzn2023.0.1.x86_64 25/35
Verifying : libXrandr-1.5.4-3.amzn2023.0.1.x86_64 26/35
Verifying : libXrender-0.9.11-6.amzn2023.0.1.x86_64 27/35
Verifying : libXt-1.3.0-3.amzn2023.0.1.x86_64 28/35
Verifying : libXtst-1.2.5-1.amzn2023.0.1.x86_64 29/35
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 30/35
Verifying : libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64 31/35
Verifying : libpng-2.1.6-37-10.amzn2023.0.13.x86_64 32/35
Verifying : libxcb-1.17.0-1.amzn2023.0.1.x86_64 33/35
Verifying : pixman-0.43.4-1.amzn2023.0.4.x86_64 34/35
Verifying : xml-common-0.6.3-56.amzn2023.0.2.noarch 35/35

Installed:
alsa-lib-1.2.7-2.1.amzn2023.0.3.x86_64
cairo-1.18.0-4.amzn2023.0.3.x86_64
dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch
dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch
dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch
fontconfig-2.13.94-2.amzn2023.0.2.x86_64
fonts-filesystem-1:2.0.5-12.amzn2023.0.2.noarch
freetype-2.13.2-5.amzn2023.0.2.x86_64
glib-5.2.1-9.amzn2023.0.4.x86_64
google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch
google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
graphite2-1.3.14-7.amzn2023.0.3.x86_64
harfbuzz-7.0.0-2.amzn2023.0.2.x86_64
java-21-amazon-corretto-1:21.0.12-8-1.amzn2023.1.x86_64
java-21-amazon-corretto-headless-1:21.0.12-8-1.amzn2023.1.x86_64
javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
libICE-1.1.1-3.amzn2023.0.1.x86_64
libSM-1.2.4-3.amzn2023.0.1.x86_64
libX11-1.8.10-2.amzn2023.0.1.x86_64
libX11-common-1.8.10-2.amzn2023.0.1.noarch
libXau-1.0.11-6.amzn2023.0.1.x86_64
libXext-1.3.6-1.amzn2023.0.1.x86_64
libXt-1.8.2-1.amzn2023.0.1.x86_64
libXinerama-1.1.5-6.amzn2023.0.1.x86_64
libXrandr-1.5.4-3.amzn2023.0.1.x86_64
libXrender-0.9.11-6.amzn2023.0.1.x86_64
libXt-1.3.0-3.amzn2023.0.1.x86_64
libXtst-1.2.5-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
libpng-2.1.6-37-10.amzn2023.0.13.x86_64
libxcb-1.17.0-1.amzn2023.0.1.x86_64
pixman-0.43.4-1.amzn2023.0.4.x86_64
xml-common-0.6.3-56.amzn2023.0.2.noarch

Complete!
[root@ip-172-31-19-86 ~]# java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode,
sharing)
[root@ip-172-31-19-86 ~]#
```

Step 3: Create Jenkins User on Agent 1

Run on Agent 1:

sudo useradd -m -s /bin/bash jenkins

Verify:

id jenkins

Step 4: Install SSH Server on Agent 1

Run on Agent 1:

sudo dnf install openssh-server -y

Start SSH:

sudo systemctl start sshd

Enable SSH:

sudo systemctl enable sshd

Verify:

sudo systemctl status sshd

It should show:

Active: active (running)

```
root@ip-172-31-19-86:~
[root@ip-172-31-19-86 ~]# id jenkins
id: 'jenkins': no such user
[root@ip-172-31-19-86 ~]# sudo useradd -m -s /bin/bash jenkins
[root@ip-172-31-19-86 ~]# id
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0-c1023
[root@ip-172-31-19-86 ~]# id jenkins
uid=1001(jenkins) gid=1001(jenkins) groups=1001(jenkins)
[root@ip-172-31-19-86 ~]# sudo dnf install openssh-server -y
Last metadata expiration check: 0:13:57 ago on Wed Aug 12 09:08:30 2026.
Package openssh-server-9.9p1-10.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-19-86 ~]# sudo systemctl start sshd
[root@ip-172-31-19-86 ~]# sudo systemctl enable sshd
[root@ip-172-31-19-86 ~]# sudo systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-08-12 09:04:20 UTC; 18min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 1887 (sshd)
     Tasks: 1 (limit: 1059)
    Memory: 5.9M
       CPU: 165ms
   CGroup: /system.slice/ssh.service
           └─1887 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 12 09:04:20 ip-172-31-19-86.ec2.internal sshd[1887]: Server listening on 0.0.0.0 port 22.
Aug 12 09:04:20 ip-172-31-19-86.ec2.internal sshd[1887]: Server listening on :: port 22.
Aug 12 09:04:20 ip-172-31-19-86.ec2.internal systemd[1]: Started sshd.service - OpenSSH server daemon.
Aug 12 09:05:48 ip-172-31-19-86.ec2.internal sshd-session[2026]: Connection closed by 194.39.227.21 port 62303.
Aug 12 09:07:37 ip-172-31-19-86.ec2.internal sshd-session[2084]: Accepted publickey for ec2-user from 103.160.27.160 port 57516 ssh2: RSA SHA256:db4L/XzwhDPhXtbQh+NU6c4EJbaS9kGN2daxbwazP5E
Aug 12 09:21:02 ip-172-31-19-86.ec2.internal sshd-session[26374]: Accepted publickey for ec2-user from 103.160.27.160 port 57516 ssh2: RSA SHA256:db4L/XzwhDPhXtbQh+NU6c4EJbaS9kGN2daxbwazP5E
Aug 12 09:21:02 ip-172-31-19-86.ec2.internal sshd-session[26374]: pam_unix(sshd:session) session opened for user ec2-user(uid=1000) by (uid=0)
Aug 12 09:21:18 ip-172-31-19-86.ec2.internal sshd-session[26443]: Connection closed by 194.39.227.21 port 62303.
Aug 12 09:21:24 ip-172-31-19-86.ec2.internal sshd-session[26444]: Connection closed by 194.39.227.21 port 62303.
[root@ip-172-31-19-86 ~]#
```

Step 5: Login to Agent 2

Connect to the second EC2 instance:

ssh -i your-key.pem ec2-user@<AGENT-2-IP>

Step 6: Check Java on Agent 2

Run:

java -version

If Java 21 is not installed:

sudo dnf install java-21-amazon-corretto -y

Verify:

java -version

```
root@ip-172-31-21-147:~  
Verifying : libXext-1.3.6-1.amzn2023.0.1.x86_64 23/35  
Verifying : libXi-1.8.2-1.amzn2023.0.1.x86_64 24/35  
Verifying : libXinerama-1.1.5-6.amzn2023.0.1.x86_64 25/35  
Verifying : libXrandr-1.5.4-3.amzn2023.0.1.x86_64 26/35  
Verifying : libXrender-0.9.11-6.amzn2023.0.1.x86_64 27/35  
Verifying : libXt-1.3.0-3.amzn2023.0.1.x86_64 28/35  
Verifying : libXtst-1.2.5-1.amzn2023.0.1.x86_64 29/35  
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 30/35  
Verifying : libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64 31/35  
Verifying : libpng-2:1.6.37-10.amzn2023.0.13.x86_64 32/35  
Verifying : libxcb-1.17.0-1.amzn2023.0.1.x86_64 33/35  
Verifying : pixman-0.43.4-1.amzn2023.0.4.x86_64 34/35  
Verifying : xml-common-0.6.3-56.amzn2023.0.2.noarch 35/35  
  
Installed:  
alsa-lib-1.2.7.2-1.amzn2023.0.3.x86_64  
cairo-1.18.0-4.amzn2023.0.3.x86_64  
dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch  
dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch  
dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch  
fontconfig-2.13.94-2.amzn2023.0.2.x86_64  
fonts-filesystem-1:2.0.5-12.amzn2023.0.2.noarch  
freetype-2.13.2-5.amzn2023.0.2.x86_64  
glib-5.2.1-9.amzn2023.0.4.x86_64  
google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch  
google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch  
graphite2-1.3.14-7.amzn2023.0.3.x86_64  
harfbuzz-7.0.0-2.amzn2023.0.2.x86_64  
java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64  
java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_64  
javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch  
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch  
libICE-1.1.1-3.amzn2023.0.1.x86_64  
libSM-1.2.4-3.amzn2023.0.1.x86_64  
libX11-1.8.10-2.amzn2023.0.1.x86_64  
libX11-common-1.8.10-2.amzn2023.0.1.noarch  
libXau-1.0.11-6.amzn2023.0.1.x86_64  
libXext-1.3.6-1.amzn2023.0.1.x86_64  
libXi-1.8.2-1.amzn2023.0.1.x86_64  
libXinerama-1.1.5-6.amzn2023.0.1.x86_64  
libXrandr-1.5.4-3.amzn2023.0.1.x86_64  
libXrender-0.9.11-6.amzn2023.0.1.x86_64  
libXt-1.3.0-3.amzn2023.0.1.x86_64  
libXtst-1.2.5-1.amzn2023.0.1.x86_64  
libbrotli-1.0.9-4.amzn2023.0.2.x86_64  
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64  
libpng-2:1.6.37-10.amzn2023.0.13.x86_64  
libxcb-1.17.0-1.amzn2023.0.1.x86_64  
pixman-0.43.4-1.amzn2023.0.4.x86_64  
xml-common-0.6.3-56.amzn2023.0.2.noarch  
  
Complete!  
[root@ip-172-31-21-147 ~]# java -version  
openjdk version "21.0.12" 2026-07-21 LTS  
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)  
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode,  
sharing)  
[root@ip-172-31-21-147 ~]#
```

700 extra

Step 7: Create Jenkins User on Agent 2

Run on Agent 2:

```
sudo useradd -m -s /bin/bash jenkins
```

Verify:

```
id jenkins
```

Step 8: Install SSH Server on Agent 2

Run on Agent 2:

```
sudo dnf install openssh-server -y
```

Start SSH:

```
sudo systemctl start sshd
```

Enable SSH:

```
sudo systemctl enable sshd
```

Verify:

sudo systemctl status sshd

```
root@ip-172-31-21-147:~# sudo useradd -m -s /bin/bash jenkins
root@ip-172-31-21-147:~# id jenkins
uid=1001(jenkins) gid=1001(jenkins) groups=1001(jenkins)
root@ip-172-31-21-147:~# sudo dnf install openssh-server -y
Last metadata expiration check: 0:05:24 ago on Wed Aug 12 09:12:13 2026.
Package openssh-server-9.9p1-10.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
root@ip-172-31-21-147:~# sudo systemctl start sshd
root@ip-172-31-21-147:~# sudo systemctl enable sshd
root@ip-172-31-21-147:~# sudo systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-08-12 09:05:08 UTC; 11min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
  Main PID: 1891 (sshd)
    Tasks: 1 (limit: 1059)
   Memory: 5.7M
      CPU: 76ms
  CGroup: /system.slice/ssh.service
          └─1891 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 12 09:05:08 ip-172-31-21-147.ec2.internal systemd[1]: Starting sshd.service: OpenSSH server daemon...
Aug 12 09:05:08 ip-172-31-21-147.ec2.internal sshd[1891]: Server listening on 0.0.0.0 port 22.
Aug 12 09:05:08 ip-172-31-21-147.ec2.internal systemd[1]: Started sshd.service: OpenSSH server daemon.
Aug 12 09:11:11 ip-172-31-21-147.ec2.internal sshd-session[2203]: Accepted publickey for ec2-user from 103.160.27.160 port 61018 ssh2: RSA SHA256:dh4L/XzmhDPhtbqh+NU6c4EJbas9kON2dAxbwazp5E
Aug 12 09:11:12 ip-172-31-21-147.ec2.internal sshd-session[2203]: pam_unix(sshd:session): session opened for user ec2-user(uid=1000) by (uid=0)
lines 13/15 (END) ... skipping...
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-08-12 09:05:08 UTC; 11min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
  Main PID: 1891 (sshd)
    Tasks: 1 (limit: 1059)
   Memory: 5.7M
      CPU: 76ms
  CGroup: /system.slice/ssh.service
          └─1891 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 12 09:05:08 ip-172-31-21-147.ec2.internal systemd[1]: Starting sshd.service: OpenSSH server daemon...
Aug 12 09:05:08 ip-172-31-21-147.ec2.internal sshd[1891]: Server listening on 0.0.0.0 port 22.
Aug 12 09:05:08 ip-172-31-21-147.ec2.internal systemd[1]: Started sshd.service: OpenSSH server daemon.
Aug 12 09:11:11 ip-172-31-21-147.ec2.internal sshd-session[2203]: Accepted publickey for ec2-user from 103.160.27.160 port 61018 ssh2: RSA SHA256:dh4L/XzmhDPhtbqh+NU6c4EJbas9kON2dAxbwazp5E
Aug 12 09:11:12 ip-172-31-21-147.ec2.internal sshd-session[2203]: pam_unix(sshd:session): session opened for user ec2-user(uid=1000) by (uid=0)
~
~
```

Agent-2

Step 10: Test SSH Connectivity

From the Jenkins Controller, test Agent 1:

```
ssh jenkins@<AGENT-1-PRIVATE-IP>
```

Then test Agent 2:

```
ssh jenkins@<AGENT-2-PRIVATE-IP>
```

If both connections work, exit:

```
exit
```

Step 11: Generate SSH Key on Controller

Login to the Jenkins Controller and run:

```
sudo -u jenkins ssh-keygen -t ed25519
```

Here:

-t → Specifies the key type

ed25519 → SSH key algorithm

When asked for the file location, press **Enter**.

When asked for a passphrase, press **Enter** twice for this lab.

Step 12: Verify SSH Key

Run:

```
sudo ls -la /var/lib/jenkins/.ssh/
```

You should see:

```
id_ed25519
```

```
id_ed25519.pub
```

Here:

```
id_ed25519    → Private key
```

```
id_ed25519.pub → Public key
```

Never share the private key.

Step 13: Copy the Public Key

Run on the Controller:

```
sudo cat /var/lib/jenkins/.ssh/id_ed25519.pub
```

Copy the complete output.

It will look similar to:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAA... jenkins@controller
```

```
root@ip-172-31-19-206:~  
[root@ip-172-31-19-206 ~]# sudo -u jenkins ssh-keygen -t ed25519  
Generating public/private ed25519 key pair.  
Enter file in which to save the key (/var/lib/jenkins/.ssh/id_ed25519):  
Created directory '/var/lib/jenkins/.ssh'.  
Enter passphrase for "/var/lib/jenkins/.ssh/id_ed25519" (empty for no passphrase  
):  
Enter same passphrase again:  
Your identification has been saved in /var/lib/jenkins/.ssh/id_ed25519  
Your public key has been saved in /var/lib/jenkins/.ssh/id_ed25519.pub  
The key fingerprint is:  
SHA256:Vf3M1TuzCPiljIEy91l684YhKXYxb+RWgEnX8aGmjXU jenkins@ip-172-31-19-206.ec2.  
internal  
The key's randomart image is:  
+--[ED25519 256]--+  
  .+o.o.o|  
  +..oo+|  
  o +.E+|  
  o o*..+=|  
  +S.o+*.o +|  
  .o=+++ * |  
  ..o++o* |  
  o..oo. |  
  ... .. |  
+-----[SHA256]-----+  
[root@ip-172-31-19-206 ~]# sudo ls -la /var/lib/jenkins/.ssh/  
total 24  
drwx----- 2 jenkins jenkins 46 Aug 12 09:47 .  
drwxr-xr-x 17 jenkins jenkins 16384 Aug 12 09:47 ..  
-rw----- 1 jenkins jenkins 432 Aug 12 09:47 id_ed25519  
-rw-r--r-- 1 jenkins jenkins 119 Aug 12 09:47 id_ed25519.pub  
[root@ip-172-31-19-206 ~]# sudo cat /var/lib/jenkins/.ssh/id_ed25519.pub  
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIHpHKDKw+APphoR9YvD1bAspxf6LwrMe6ECz8j18Gdk3  
jenkins@ip-172-31-19-206.ec2.internal  
[root@ip-172-31-19-206 ~]#
```

Step 14: Add Public Key to Agent 1

```
sudo su - jenkins
```

Create SSH directory:

```
mkdir -p ~/.ssh
```

Set permission:

```
chmod 700 ~/.ssh
```

Open:

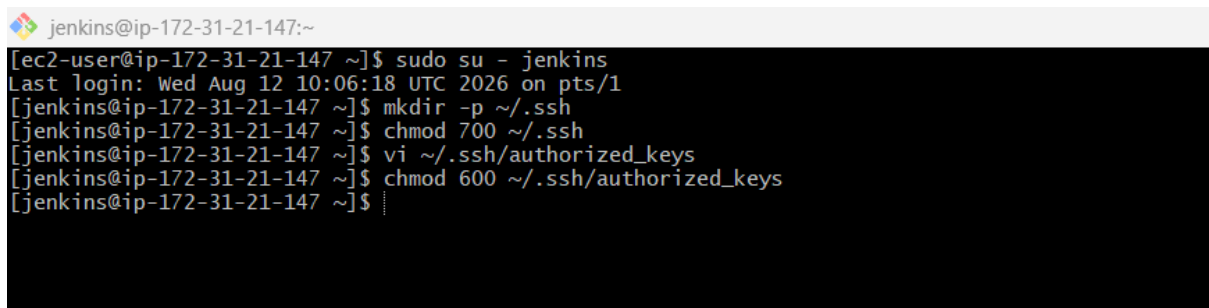
```
vi ~/.ssh/authorized_keys
```

Paste the same public key.

Save and exit.

Set permission:

```
chmod 600 ~/.ssh/authorized_keys
```

A terminal window screenshot showing the configuration of passwordless SSH for Jenkins. The prompt is 'jenkins@ip-172-31-21-147:~'. The user runs 'sudo su - jenkins', which shows the last login as 'Wed Aug 12 10:06:18 UTC 2026 on pts/1'. Then, the user runs 'mkdir -p ~/.ssh', 'chmod 700 ~/.ssh', 'vi ~/.ssh/authorized_keys', and 'chmod 600 ~/.ssh/authorized_keys'. The terminal ends with a prompt 'jenkins@ip-172-31-21-147 ~]\$'.

```
jenkins@ip-172-31-21-147:~  
[ec2-user@ip-172-31-21-147 ~]$ sudo su - jenkins  
Last login: Wed Aug 12 10:06:18 UTC 2026 on pts/1  
[jenkins@ip-172-31-21-147 ~]$ mkdir -p ~/.ssh  
[jenkins@ip-172-31-21-147 ~]$ chmod 700 ~/.ssh  
[jenkins@ip-172-31-21-147 ~]$ vi ~/.ssh/authorized_keys  
[jenkins@ip-172-31-21-147 ~]$ chmod 600 ~/.ssh/authorized_keys  
[jenkins@ip-172-31-21-147 ~]$
```

Step 16: Test Passwordless SSH to Agent 1

On the Controller:

```
sudo -u jenkins ssh jenkins@<AGENT-1-PRIVATE-IP>
```

The Controller should connect without asking for the Jenkins password.

Exit:

```
exit
```

Step 17: Test Passwordless SSH to Agent 2

Run:

```
sudo -u jenkins ssh jenkins@<AGENT-2-PRIVATE-IP>
```

The Controller should connect without asking for the password.

exit

```
jenkins@ip-172-31-21-147::~~$  
brajk@RajKumari MINGW64 ~/Downloads  
$ ssh -i "linux-testing.pem" ec2-user@98.88.84.114 controller/Master Jenkins  
  
#  
#####  
Amazon Linux 2023  
#####  
|  
##  
/#/ https://aws.amazon.com/linux/amazon-linux-2023  
V~/  
^-->  
_/_/  
m/'  
Last login: Wed Aug 12 10:09:43 2026 from 103.160.27.160  
[ec2-user@ip-172-31-19-206 ~]$ sudo -u jenkins ssh jenkins@172.31.19.86  
The authenticity of host '172.31.19.86 (172.31.19.86)' can't be established.  
ED25519 key fingerprint is SHA256:UfKTtOshdBzUGc6ThsFY8KDF/C8uPujXlyFkvvoEXI.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '172.31.19.86' (ED25519) to the list of known hosts.  
  
#  
#####  
Amazon Linux 2023  
#####  
|  
##  
/#/ https://aws.amazon.com/linux/amazon-linux-2023  
V~/  
^-->  
_/_/  
m/'  
Last login: Wed Aug 12 09:59:24 2026  
[jenkins@ip-172-31-19-86 ~]$ exit  
logout  
Connection to 172.31.19.86 closed.  
[ec2-user@ip-172-31-19-206 ~]$ sudo -u jenkins ssh jenkins@172.31.21.147  
The authenticity of host '172.31.21.147 (172.31.21.147)' can't be established.  
ED25519 key fingerprint is SHA256:xMdyqGZFxFZfn6YAQ6zpU8YOGH2gUja5gvHF+BC3czlw.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '172.31.21.147' (ED25519) to the list of known hosts.  
  
#  
#####  
Amazon Linux 2023  
#####  
|  
##  
/#/ https://aws.amazon.com/linux/amazon-linux-2023  
V~/  
^-->  
_/_/  
m/'  
Last login: Wed Aug 12 10:07:10 2026  
[jenkins@ip-172-31-21-147 ~]$ |
```

Step 18: Open Jenkins Dashboard

Open Jenkins in your browser:

http://<CONTROLLER-IP>:8080

Login with your Jenkins administrator account.

Step 19: Open Nodes

Go to:

Jenkins Dashboard → Manage Jenkins → Nodes

Click:

New Node

Step 20: Create Agent 1

Enter:

Name: agent-1

Select:

Permanent Agent

Click:

Create

Step 21: Configure Agent 1

Enter:

Number of executors: 1

Enter:

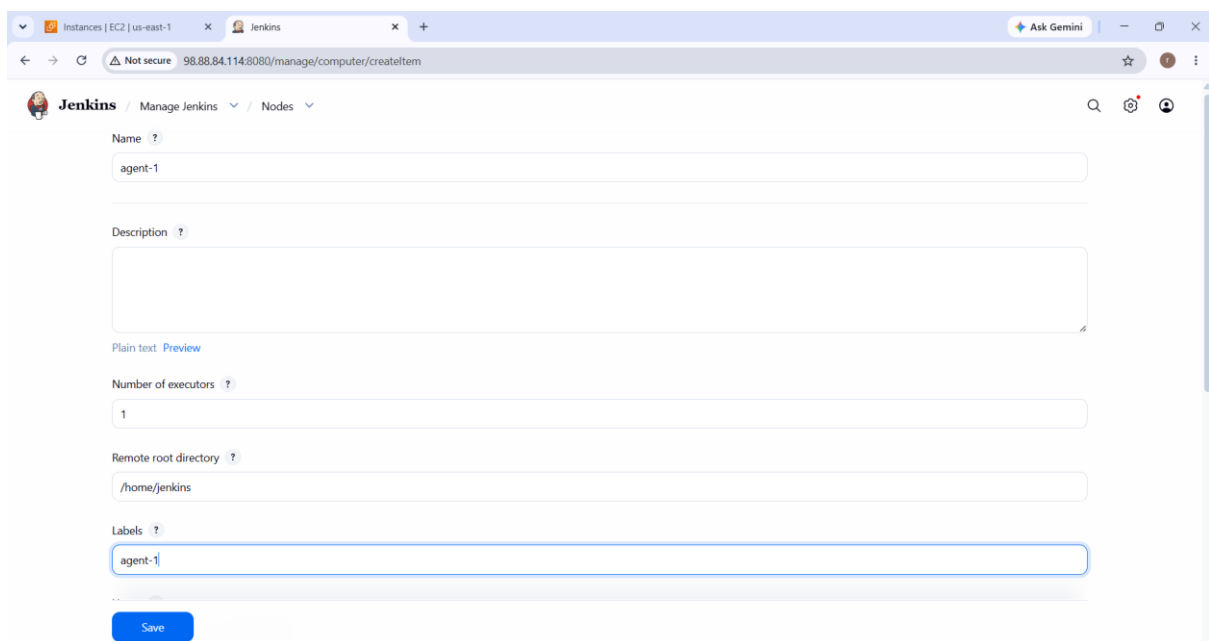
Remote root directory: /home/jenkins

Enter label:

agent1

Select:

Use this node as much as possible



The screenshot shows the Jenkins web interface in a browser. The address bar indicates the URL is `98.88.84.114:8080/manage/computer/createItem`. The page title is "Jenkins" and the breadcrumb navigation shows "Manage Jenkins" > "Nodes". The form is for creating a new node and includes the following fields:

- Name**: `agent-1`
- Description**: (Empty text area)
- Number of executors**: `1`
- Remote root directory**: `/home/jenkins`
- Labels**: `agent-1`

At the bottom of the form is a blue "Save" button. The browser's developer tools are open on the right side of the screen.

Step 22: Configure SSH for Agent 1

Under **Launch method**, select:

Launch agents via SSH

Enter Agent 1's private IP:

<AGENT-1-PRIVATE-IP>

Under **Credentials**, click:

Add → Jenkins

Select:

Kind: SSH Username with private key

Username: jenkins

Private Key: Enter directly

On the Controller, get the private key:

```
sudo cat /var/lib/jenkins/.ssh/id_ed25519
```

Copy it and paste it into the credential.

Click **Add**.

Select the created credential.

Step 23: Save Agent 1

Click:

Save

Go to:

Manage Jenkins → Nodes → agent-1

Verify that Agent 1 shows:

Connected

Jenkins / Nodes / agent-1 / Log

- Status
- Delete Agent
- Configure
- Build History
- Load Statistics
- Script Console
- Log**
- System Information
- Disconnect

Build Executor Status ^

(0 of 1 executor busy)

```
Warning: no key algorithms provided; JENKINS-42959 disabled
SSHLauncher{host='172.31.19.86', port=22, credentialsId='jenkins-agent-ssh',
jvmOptions='', javaPath='', prefixStartSlaveCmd='', suffixStartSlaveCmd='',
launchTimeoutSeconds=60, maxNumRetries=10, retryWaitTime=15,
sshHostKeyVerificationStrategy=udson.plugins.sshslaves.verifiers.KnownHostsFileKeyVerif
icationStrategy, tcpNoDelay=true, trackCredentials=true}
100/10/20 11:20:20.500 [SSHLauncher] INFO: javaPath: /usr/bin/java
```

Step 24: Create Agent 2

Go to:

Manage Jenkins → Nodes → New Node

Enter:

Name: agent-2

Select:

Permanent Agent

Click:

Create

Step 25: Configure Agent 2

Enter:

Number of executors: 1

Remote root directory: /home/jenkins

Labels: agent2

Select:

Use this node as much as possible

Step 26: Configure SSH for Agent 2

Under **Launch method**, select:

Launch agents via SSH

Enter:

Host: <AGENT-2-PRIVATE-IP>

Select the SSH credential created earlier.

Click:

Save

Step 27: Verify Agent 2

Go to:

Manage Jenkins → Nodes → agent-2

Verify:

Connected

```
which ()
{
    ( alias;
      eval ${which_declare} ) | /usr/bin/which --tty-only --read-alias --read-functions --
show-tilde --show-dot "$@"
}
[08/12/26 11:35:45] [SSH] Starting sftp client.
[08/12/26 11:35:45] [SSH] Copying latest remoting.jar...
[08/12/26 11:35:45] [SSH] Copied 1,407,938 bytes.
Expanded the channel window size to 4MB
[08/12/26 11:35:45] [SSH] Starting agent process: cd "/home/jenkins" && java -jar
remoting.jar -workDir /home/jenkins -jar-cache /home/jenkins/remoting/jarCache
Aug 12, 2026 11:35:45 AM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using /home/jenkins/remoting as a remoting work directory
Aug 12, 2026 11:35:45 AM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to /home/jenkins/remoting
<===[JENKINS REMOTING CAPACITY]==>channel started
Remoting version: 3355.3357.v931d3c992987
Launcher: SSHLauncher
Communication Protocol: Standard in/out
This is a Unix agent
Agent successfully connected and online
```

Step 28: Verify Both Agents

Go to:

Manage Jenkins → Nodes

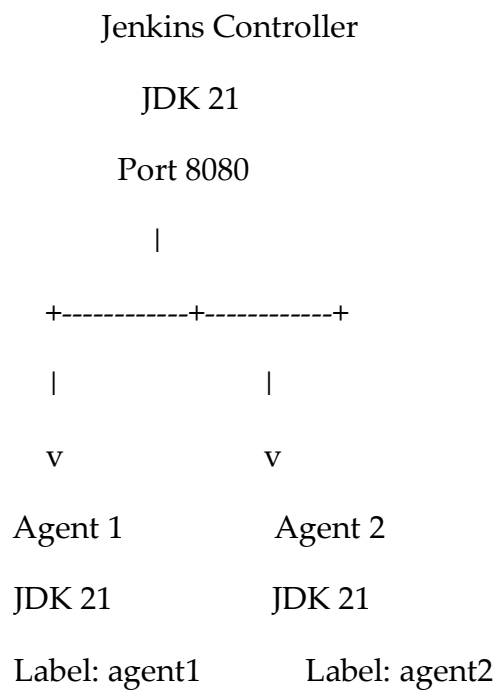
You should see:

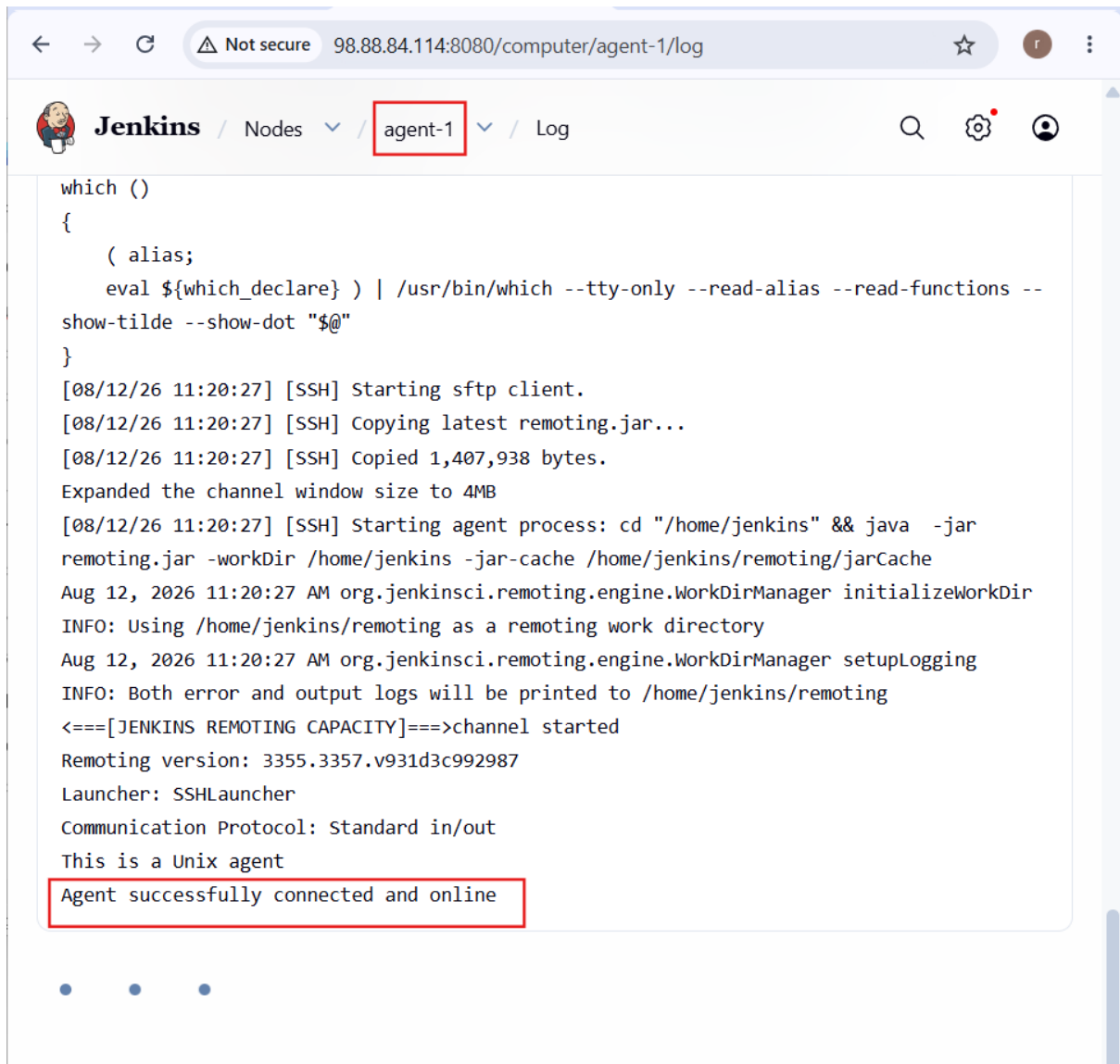
Built-In Node

agent-1 Connected

agent-2 Connected

Your architecture is now:





← → ↻ ⚠ Not secure 98.88.84.114:8080/computer/agent-1/log ☆ Ⓡ ⋮

 **Jenkins** / Nodes ▾ / **agent-1** ▾ / Log 🔍 ⚙️ 👤

```
which ()
{
    ( alias;
      eval ${which_declare} ) | /usr/bin/which --tty-only --read-alias --read-functions --
show-tilde --show-dot "$@"
    }
[08/12/26 11:20:27] [SSH] Starting sftp client.
[08/12/26 11:20:27] [SSH] Copying latest remoting.jar...
[08/12/26 11:20:27] [SSH] Copied 1,407,938 bytes.
Expanded the channel window size to 4MB
[08/12/26 11:20:27] [SSH] Starting agent process: cd "/home/jenkins" && java -jar
remoting.jar -workDir /home/jenkins -jar-cache /home/jenkins/remoting/jarCache
Aug 12, 2026 11:20:27 AM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using /home/jenkins/remoting as a remoting work directory
Aug 12, 2026 11:20:27 AM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to /home/jenkins/remoting
<===[JENKINS REMOTING CAPACITY]==>channel started
Remoting version: 3355.3357.v931d3c992987
Launcher: SSHLauncher
Communication Protocol: Standard in/out
This is a Unix agent
Agent successfully connected and online
```

• • •

Step 30: Restrict Job to Agent 1

Select:

Restrict where this project can be run

Enter:

agent1

Under **Build Steps**, select:

Execute shell

Enter:

echo "Job executed on Agent 1"

hostname

java -version

Click:

Step 31: Verify Agent 1 Job

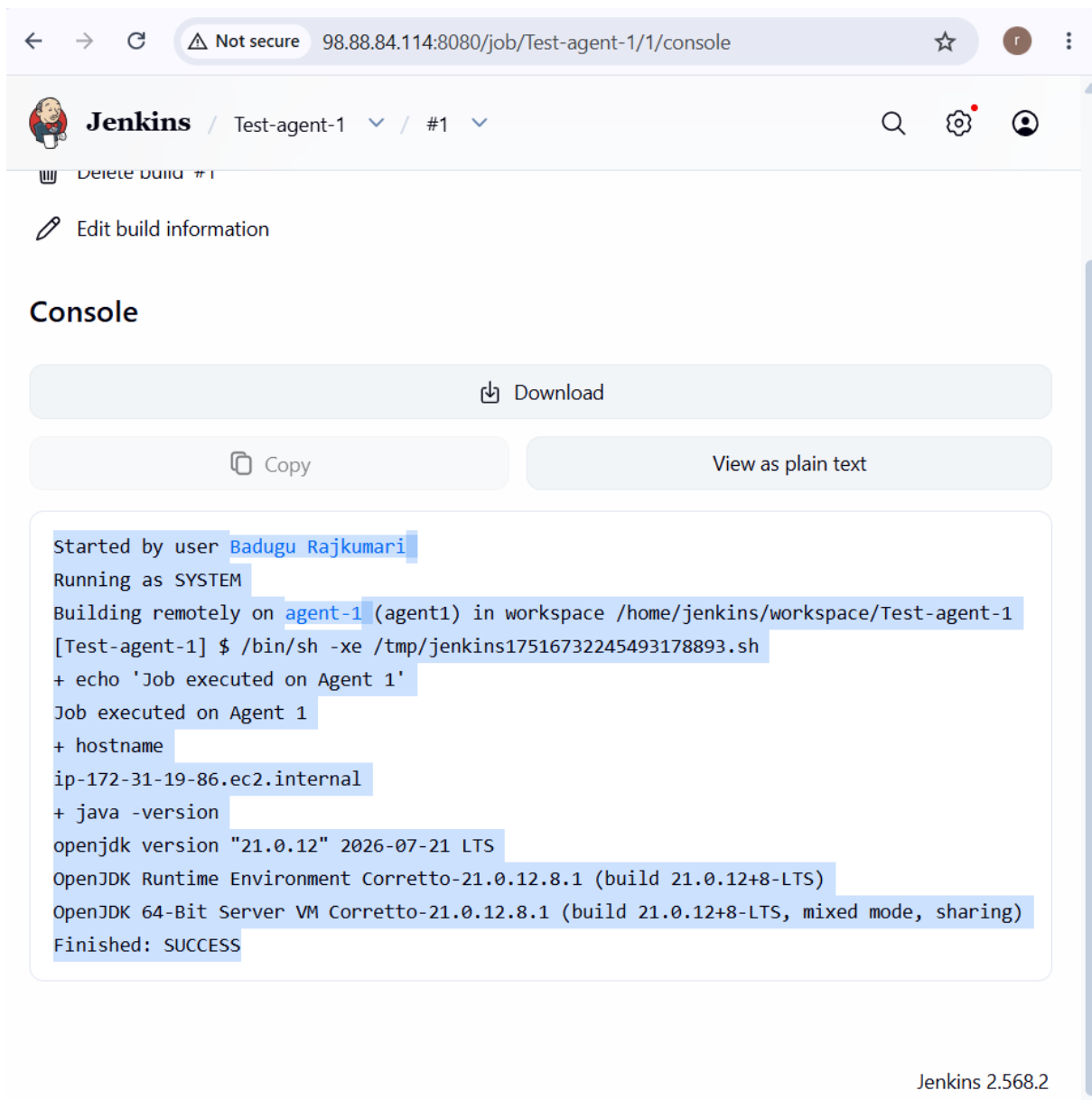
Go to:

Build #1 → Console Output

Verify:

Job executed on Agent 1

You should also see the Agent 1 hostname and Java 21 version.



The screenshot shows the Jenkins web interface for a job named 'Test-agent-1'. The browser address bar indicates the URL '98.88.84.114:8080/job/Test-agent-1/1/console'. The Jenkins header shows the logo and the job name 'Test-agent-1' with a dropdown arrow. Below the header, there are links for 'Delete build #1' and 'Edit build information'. The main section is titled 'Console' and contains a text area with the following output:

```
Started by user Badugu Rajkumari
Running as SYSTEM
Building remotely on agent-1 (agent1) in workspace /home/jenkins/workspace/Test-agent-1
[Test-agent-1] $ /bin/sh -xe /tmp/jenkins17516732245493178893.sh
+ echo 'Job executed on Agent 1'
Job executed on Agent 1
+ hostname
ip-172-31-19-86.ec2.internal
+ java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
Finished: SUCCESS
```

Below the console output, there are buttons for 'Download', 'Copy', and 'View as plain text'. The Jenkins version 'Jenkins 2.568.2' is displayed in the bottom right corner.

Step 32: Create Test Job for Agent 2

Create another job:

Test-Agent-2

Select:

Freestyle project

Click OK.

Step 33: Restrict Job to Agent 2

Select:

Restrict where this project can be run

Enter:

agent2

Under **Build Steps** → **Execute shell**, enter:

echo "Job executed on Agent 2"

hostname

java -version

Click:

Save → **Build Now**

Step 34: Verify Agent 2 Job

Go to:

Build #1 → **Console Output**

Verify:

Job executed on Agent 2

You should also see the Agent 2 hostname and Java 21 version.

← → ↻ Not secure 98.88.84.114:8080/job/Test-agent-2/1/console ☆

Jenkins / Test-agent-2 ▾ / #1 ▾ 🔍 ⚙️ 👤

🗑️ Delete build #1

✎ Edit build information

Console

📄 Download

📄 Copy View as plain text

```

Started by user Badugu Rajkumari
Running as SYSTEM
Building remotely on agent-2 (agent2) in workspace /home/jenkins/workspace/Test-agent-2
[Test-agent-2] $ /bin/sh -xe /tmp/jenkins14314705654323175413.sh
+ echo 'Job executed on Agent 2'
Job executed on Agent 2
+ hostname
ip-172-31-21-147.ec2.internal
+ java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
Finished: SUCCESS

```

Jenkins 2.569.2

Step 35: Final Result

The final setup should be:

Jenkins Controller

JDK 21

Port 8080

|

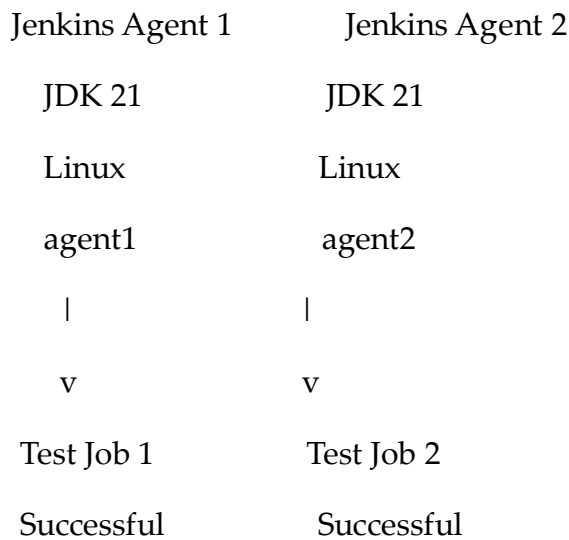
+-----+-----+

|

|

v

v



2. Configure webhooks to the Jenkins job.

Objective: To configure a GitHub webhook with Jenkins so that whenever new code is pushed to the GitHub repository, GitHub automatically notifies Jenkins and Jenkins triggers the configured job.

Flow

Developer

|
| **git push**

↓

GitHub Repository

|
| **Webhook**

↓

Jenkins Controller

|
| **Trigger Job**

↓

Jenkins Job

|



Build / Test / Deploy

Step 1: Make Sure Your Jenkins Job Exists

Go to:

Jenkins Dashboard → Your Job

For example:

Jenkins Dashboard



Jenkins-Private-App

If you don't have a job yet:

New Item → Enter item name → Freestyle project → OK

← → ↻ ⚠ Not secure 98.88.84.114:8080/view/all/newJob ☆

Jenkins / All ▾ / New Item 🔍 ⚙️ 👤

New Item

Enter an item name

Jenkins-Private-App

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Step 2: Configure Git Repository

Open your Jenkins job:

Job → Configure

Scroll to:

Source Code Management

Select:

Git

Enter your GitHub repository URL.

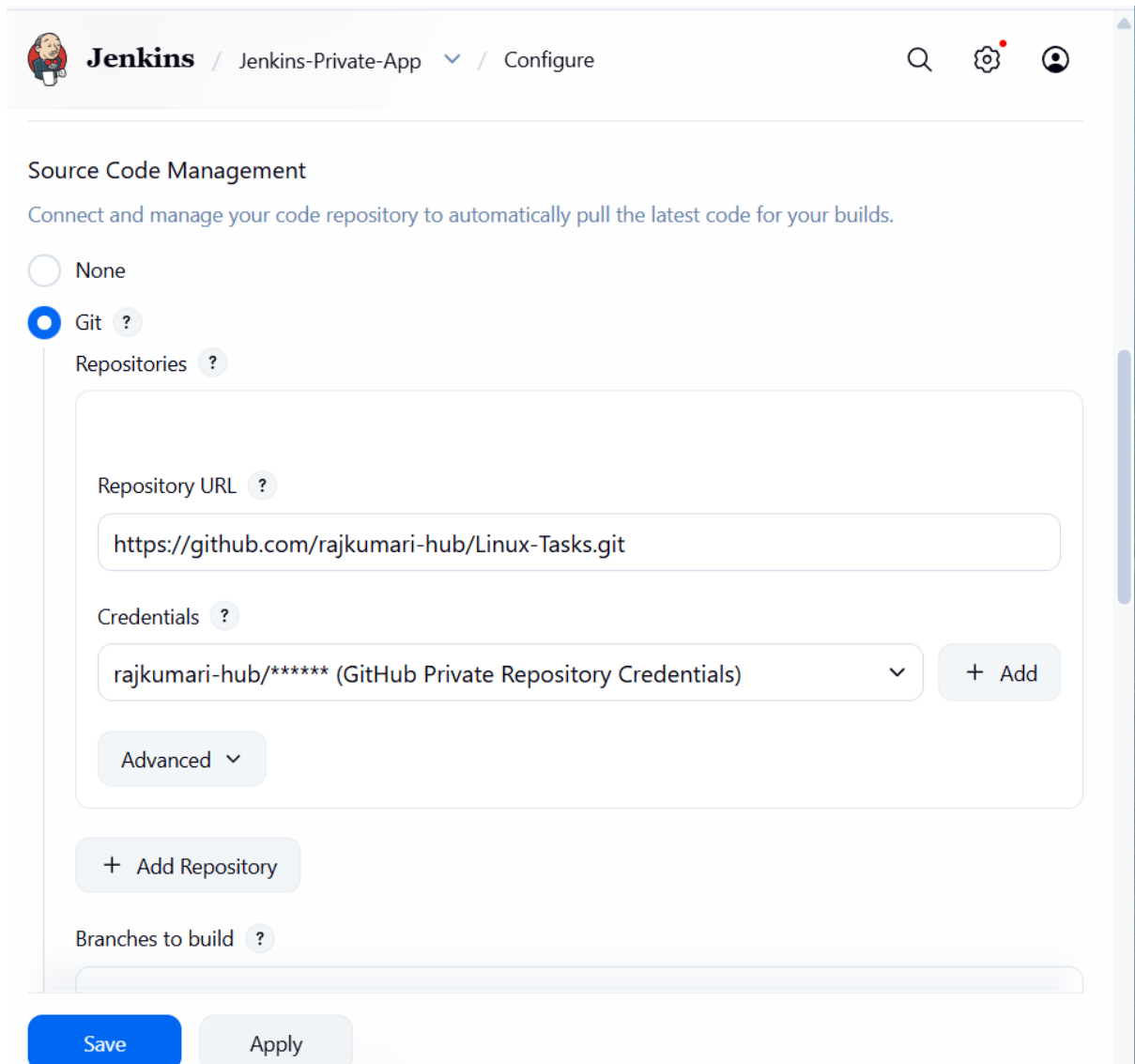
For example:

<https://github.com/rajkumari-hub/Jenkins-Private-App.git>

If the repository is private, select the appropriate GitHub credentials.

Click:

Save or continue configuring.



The screenshot shows the Jenkins web interface. At the top, the breadcrumb navigation reads 'Jenkins / Jenkins-Private-App / Configure'. The 'Source Code Management' section is active, with a description: 'Connect and manage your code repository to automatically pull the latest code for your builds.' Two radio buttons are present: 'None' (unselected) and 'Git' (selected). Below the 'Git' option is a 'Repositories' section. It contains a 'Repository URL' field with the value 'https://github.com/rajkumari-hub/Linux-Tasks.git', a 'Credentials' dropdown menu showing 'rajkumari-hub/***** (GitHub Private Repository Credentials)', and an 'Advanced' expandable section. At the bottom of the 'Repositories' section is a '+ Add Repository' button. Below this is a 'Branches to build' section with an empty text input field. At the very bottom of the configuration page are two buttons: 'Save' (in blue) and 'Apply' (in light blue).

Step 3: Configure Build Trigger

In the same job configuration page, find:

Build Triggers

Select:

GitHub hook trigger for GITScm polling

You should see a checkbox similar to:

☒ GitHub hook trigger for GITScm polling

Step 4: Configure Build Step

Scroll down to:

Build Steps

For a simple test, select:

Execute shell

Enter:

```
echo "GitHub Webhook Triggered Successfully"
```

```
echo "Build executed by Jenkins"
```

You can later replace this with your actual build commands.

Step 5: Save the Jenkins Job

Click:

Save

Your Jenkins job is now configured to accept GitHub webhook events.

The screenshot shows the Jenkins configuration interface for a project named 'Jenkins-Private-App'. The top navigation bar includes the Jenkins logo, the project name, and a 'Configure' link. On the right, there are icons for search, settings, and a user profile. Below the navigation bar, there are two checkboxes: 'GitHub hook trigger for GITScm polling' (checked) and 'Poll SCM'. The 'Environment' section contains a description and six checkboxes: 'Delete workspace before build starts', 'Use secret text(s) or file(s)', 'Add timestamps to the Console Output', 'Inspect build log for published build scans', 'Terminate a build if it's stuck', and 'With Ant'. The 'Build Steps' section has a description and a single step named 'Execute shell' with a command field containing 'echo "GitHub Webhook Triggered Successfully"'. At the bottom, there are 'Save' and 'Apply' buttons.

Jenkins / Jenkins-Private-App / Configure

☒ GitHub hook trigger for GITScm polling ?
☐ Poll SCM ?

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

☐ Delete workspace before build starts
☐ Use secret text(s) or file(s) ?
☐ Add timestamps to the Console Output
☐ Inspect build log for published build scans
☐ Terminate a build if it's stuck
☐ With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

⋮ Execute shell ?

Command
See [the list of available environment variables](#)

```
echo "GitHub Webhook Triggered Successfully"
```

Save Apply

Step 6: Find Your Jenkins Public URL

Your Jenkins Controller must be reachable from GitHub.

For example, your Jenkins URL might be:

`http://98.88.84.114:8080`

Therefore, the webhook URL will be:

`http://98.88.84.114:8080/github-webhook/`

Important

The endpoint must end with:

`/github-webhook/`

It is **not** your job URL.

✗ Incorrect:

<http://98.88.84.114:8080/job/Jenkins-Private-App/>

✓ Correct:

<http://98.88.84.114:8080/github-webhook/>

Step 7: Configure the Webhook in GitHub

Open your GitHub repository.

Go to:

GitHub → Repository → Settings

Then select:

Webhooks

Click:

Add webhook

Step 8: Enter Payload URL

In:

Payload URL

Enter:

<http://98.88.84.114:8080/github-webhook/>

Use **your actual Jenkins public IP/DNS**, not the private IP such as:

172.31.19.206

GitHub cannot normally reach an EC2 private IP over the public Internet.

Step 9: Select Content Type

Under:

Content type

Select:

application/json

Step 10: Configure Which Events Trigger Jenkins

Under:

Which events would you like to trigger this webhook?

For a simple assignment, select:

Just the push event

This means:

git push

↓

GitHub webhook

↓

Jenkins

↓

Build

You can leave:

☐ Send me everything

unselected.

Step 11: Enable SSL Verification

If your Jenkins URL is:

http://...

there is no HTTPS certificate to verify.

For a production Jenkins server using HTTPS, SSL verification should normally remain enabled.

For your current AWS lab setup using HTTP, you can use the HTTP webhook URL.

Step 12: Add the Webhook

Click:

Add webhook

GitHub will create the webhook.

You should see something similar to:

Payload URL:

`http://98.88.84.114:8080/github-webhook/`

Content type:

`application/json`

Events:

Just the push event

we'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

http://98.88.84.114:8080/github-webhook/

Content type *

application/json

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

☒ Enable SSL verification ☐ Disable (not recommended)

Which events would you like to trigger this webhook?

☒ Just the push event.

☐ Send me **everything**.

☐ Let me select individual events.

☒ **Active**

We will deliver event details when this hook is triggered.

Add webhook

Step 13: Check the Webhook

In GitHub:

Repository → Settings → Webhooks

Click the webhook you created.

You should see:

Recent Deliveries

GitHub will show webhook delivery information.

Step 14: Test the Webhook

Go to your local Git repository.

Make a small change.

For example, modify:

index.html

Then run:

```
git add .
```

```
git commit -m "Test Jenkins webhook"
```

Then:

```
git push origin main
```

If your branch is master, use:

```
git push origin master
```

Step 15: Check GitHub Webhook Delivery

Go to:

GitHub → Repository → Settings → Webhooks → Your Webhook

Open:

Recent Deliveries

You should see a new push event.

A successful delivery should show an HTTP response such as:

200

Step 16: Check Jenkins

Go to:

Jenkins Dashboard → Your Job

You should see a new build automatically triggered.

For example:

Build #1

Build #2

Build #3

The important point is that **you did not manually click Build Now**.

The sequence was:

git push



GitHub



Webhook



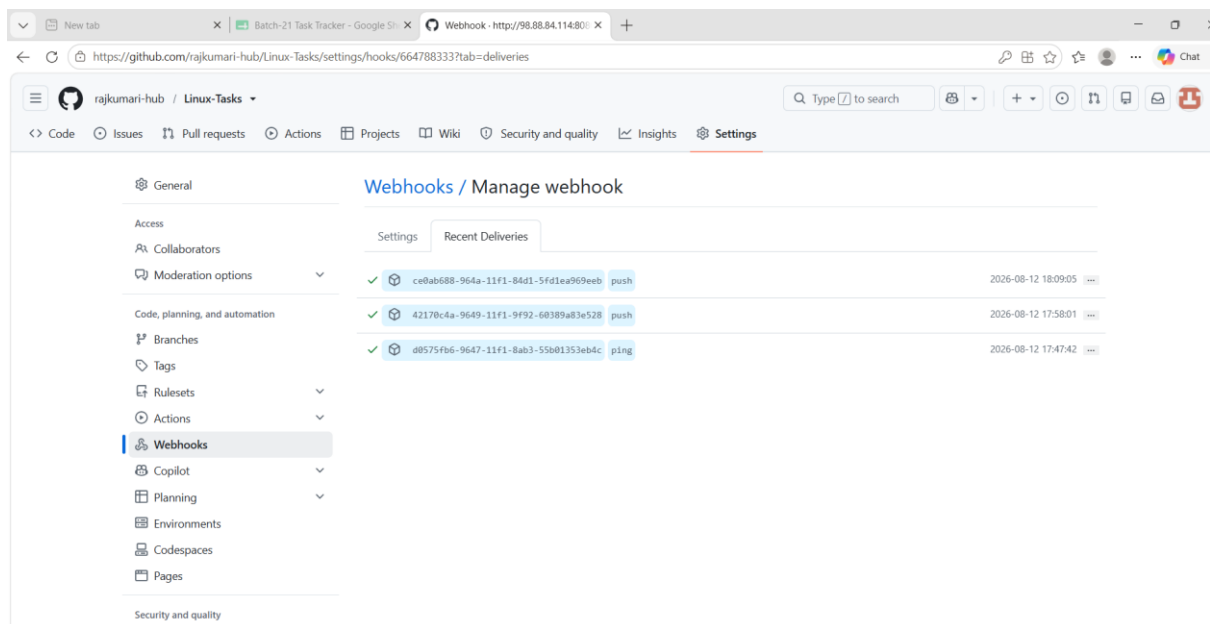
Jenkins Controller

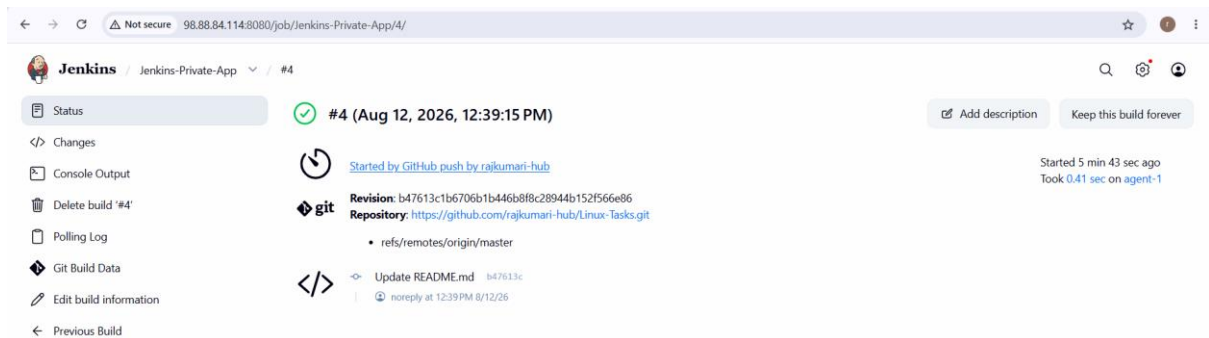
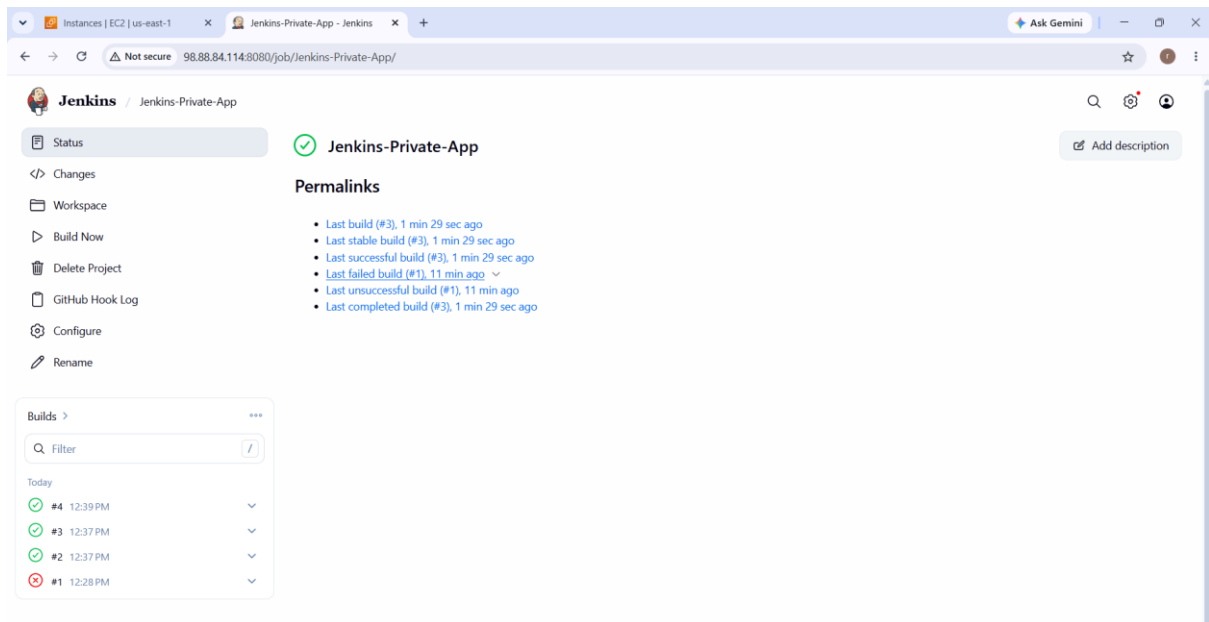


Jenkins Job



Build





Step 17: Verify Console Output

Open the newly triggered build:

Jenkins → Job → Build Number → Console Output

You should see:

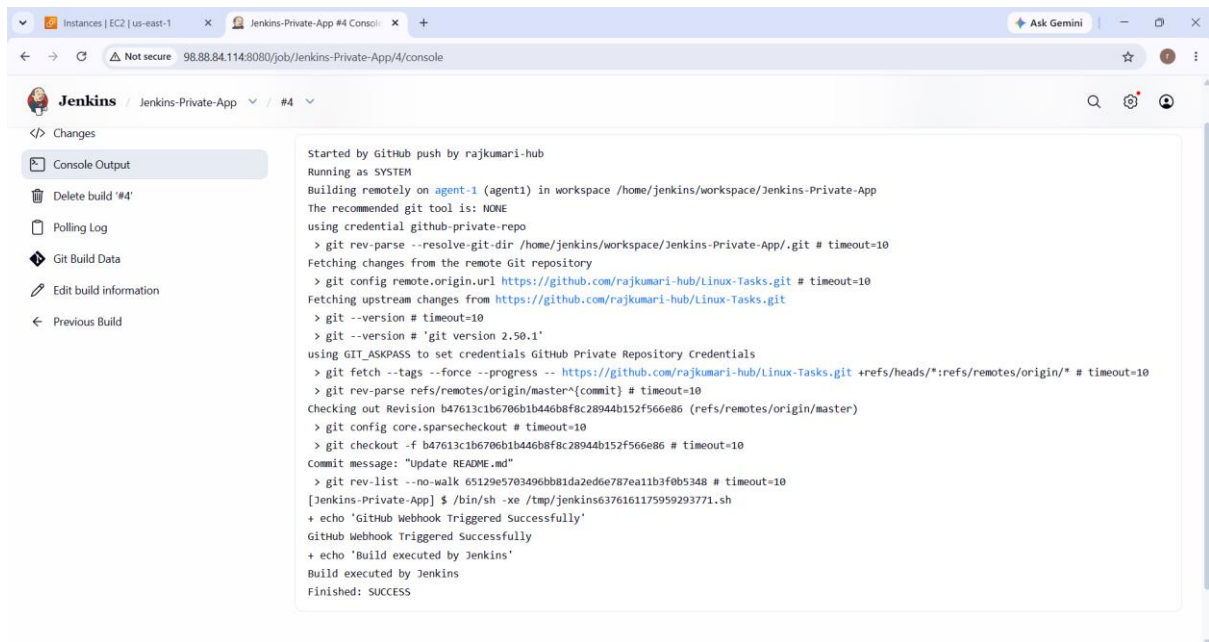
Started by GitHub push by <username>

and your build commands:

GitHub Webhook Triggered Successfully

Build executed by Jenkins

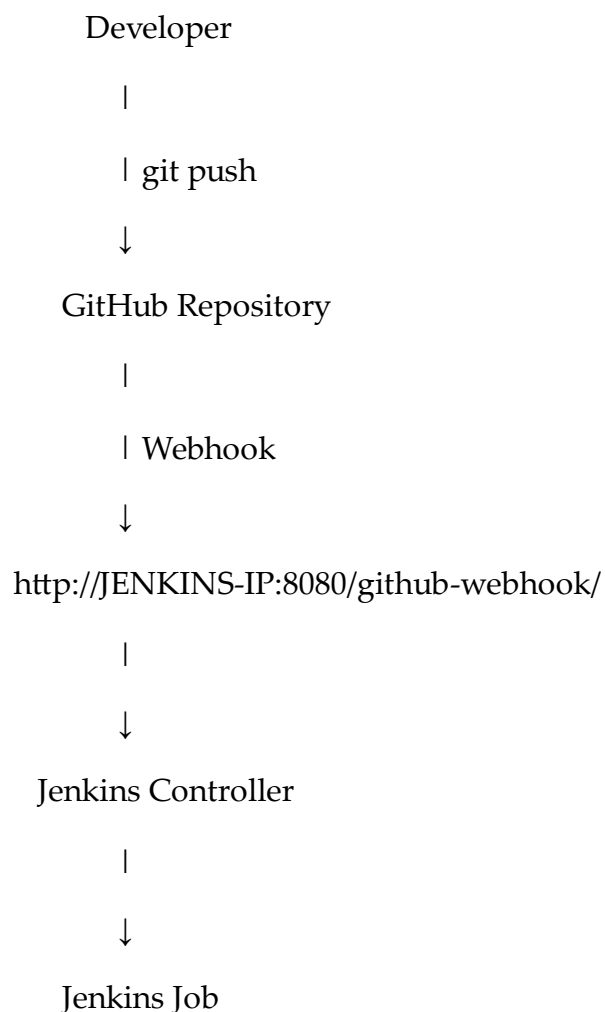
This confirms that the webhook is working.



The screenshot shows a Jenkins console window for a build named 'Jenkins-Private-App #4'. The console output details the build process, starting with a GitHub push by 'rajkumari-hub'. It shows the build running as 'SYSTEM' on 'agent-1' in the workspace '/home/jenkins/workspace/Jenkins-Private-App'. The build uses 'git' to fetch changes from the remote repository 'https://github.com/rajkumari-hub/Linux-Tasks.git'. It then checks out revision 'b47613c1b6706b1b446b8f8c28944b152f566e86' (refs/remotes/origin/master). The build message is 'Update README.md'. The build is triggered successfully by Jenkins, and the final status is 'Finished: SUCCESS'.

Step 18: Final Verification

Your final setup should look like:



|

↓

3.Configure Poll SCM and Build Periodically options in the Jenkins job.

Objective: To configure **Poll SCM** and **Build Periodically** triggers in a Jenkins job and understand the difference between them.

Part A: Configure Poll SCM

Step 1: Open the Jenkins Job

Go to:

Jenkins Dashboard → Jenkins-Private-App

Click:

Configure

Step 2: Find Build Triggers

Scroll down to:

Build Triggers

You will see options such as:

☐ GitHub hook trigger for GITScm polling

☐ Poll SCM

☐ Build periodically

Step 3: Select Poll SCM

Select:

☒ Poll SCM

A **Schedule** field will appear.

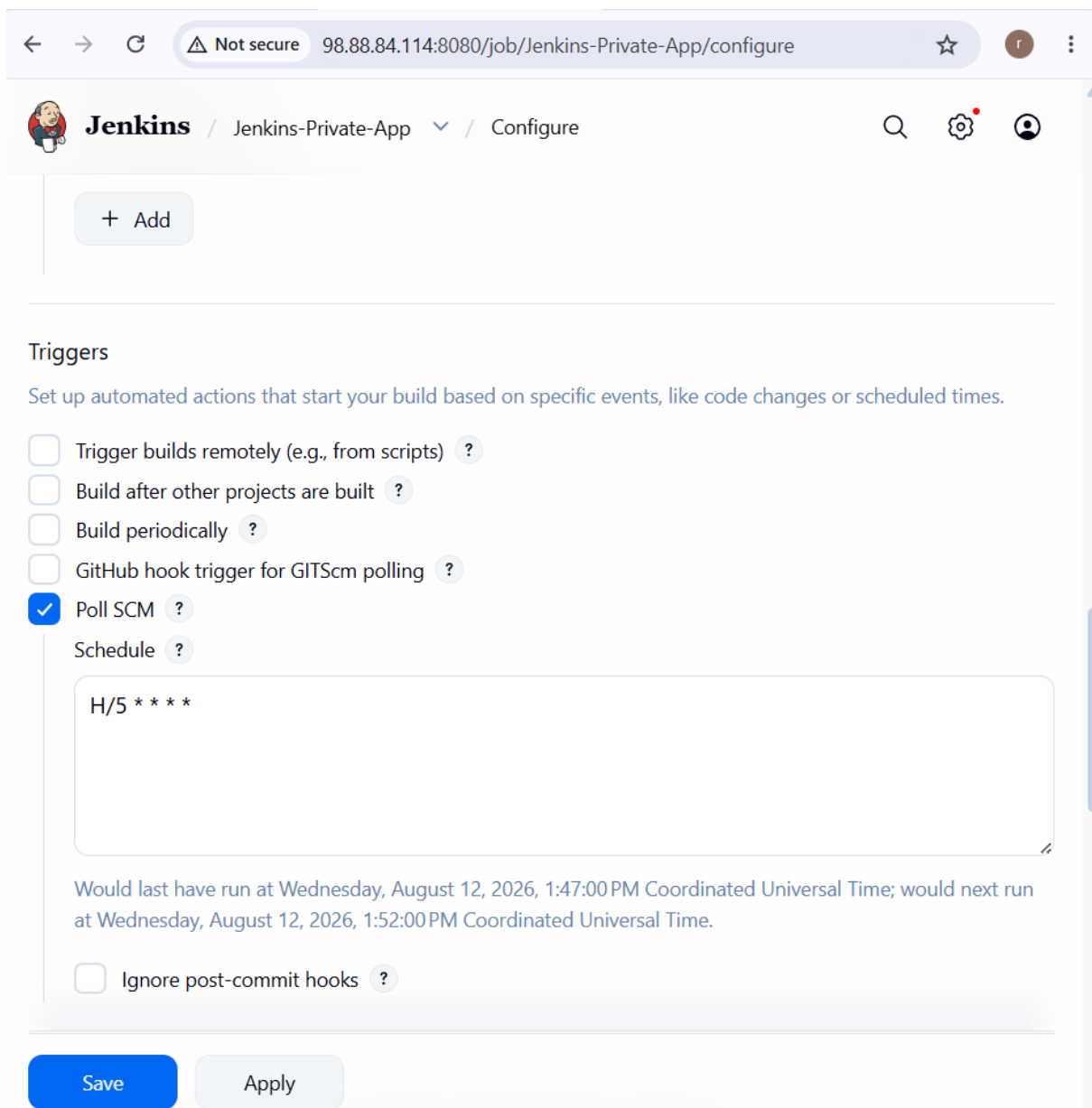
Step 4: Enter Poll SCM Schedule

For testing, enter:

`* / 5 * * * *`

This means:

Jenkins checks the Git repository every **5 minutes** for new changes.



The screenshot shows the Jenkins web interface in a browser. The address bar indicates the URL is `98.88.84.114:8080/job/Jenkins-Private-App/configure`. The page title is 'Jenkins / Jenkins-Private-App / Configure'. Below the title bar, there is a '+ Add' button. The 'Triggers' section is expanded, showing a list of triggers with checkboxes. The 'Poll SCM' option is selected with a blue checkmark. Below the 'Poll SCM' checkbox, the 'Schedule' field is visible, containing the text `H/5 * * * *`. A message below the schedule field states: 'Would last have run at Wednesday, August 12, 2026, 1:47:00 PM Coordinated Universal Time; would next run at Wednesday, August 12, 2026, 1:52:00 PM Coordinated Universal Time.' At the bottom of the configuration page, there are two buttons: 'Save' (in blue) and 'Apply' (in light gray).

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☐ GitHub hook trigger for GITScm polling ?
- ☒ Poll SCM ?

Schedule ?

`H/5 * * * *`

Would last have run at Wednesday, August 12, 2026, 1:47:00 PM Coordinated Universal Time; would next run at Wednesday, August 12, 2026, 1:52:00 PM Coordinated Universal Time.

☐ Ignore post-commit hooks ?

Save Apply

Step 5: Understand the Cron Format

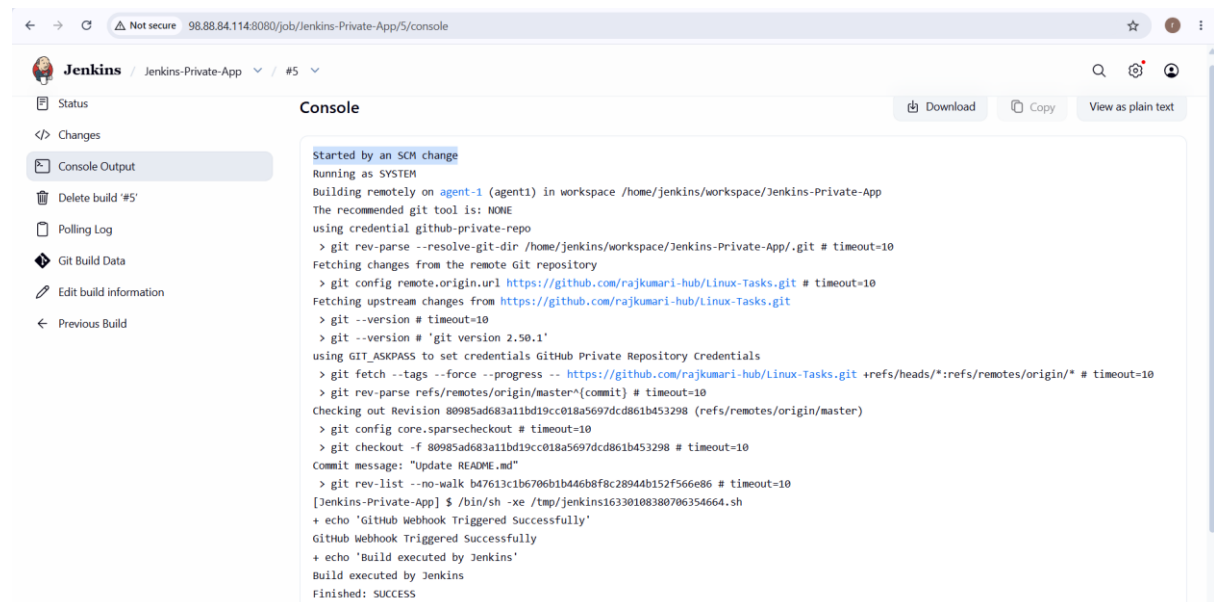
Jenkins uses five fields:

MINUTE HOUR DAY MONTH WEEK

For:

`*/5 * * * *`

the meaning is:



Field Value Meaning

Minute	<code>*/5</code>	Every 5 minutes
Hour	<code>*</code>	Every hour
Day	<code>*</code>	Every day
Month	<code>*</code>	Every month
Week	<code>*</code>	Every day of week

Important

Poll SCM does not necessarily build every time it checks.

The process is:

Every 5 minutes



Jenkins checks GitHub



Changes found?

✓ ✗

YES NO

↓ ↓

Build Nothing

So if there is **no new commit**, Jenkins does not start a build.

Part B: Configure Build Periodically

Step 6: Select Build Periodically

In the same **Build Triggers** section, select:

☒ Build periodically

A second **Schedule** field will appear.

Step 7: Enter the Schedule

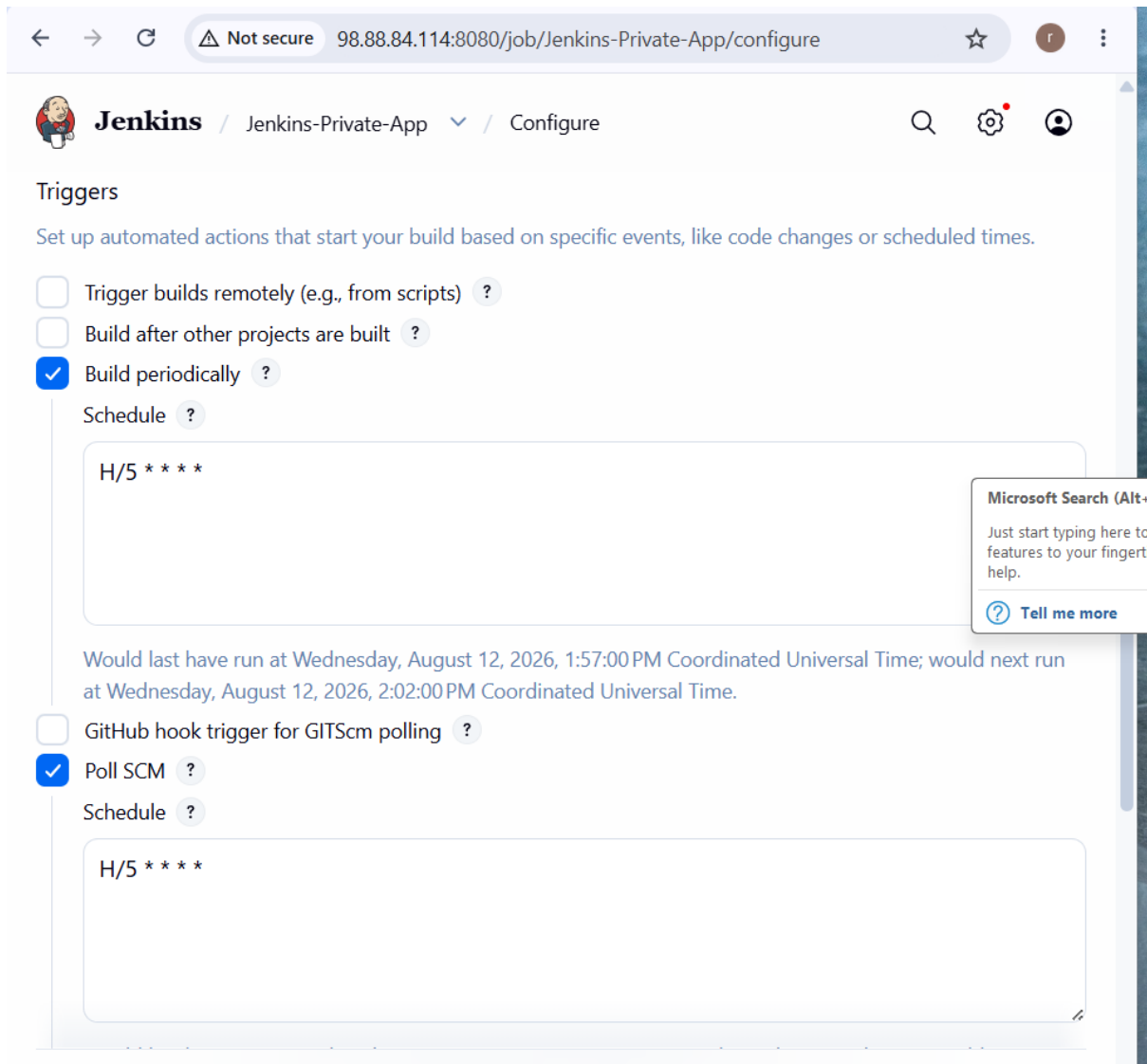
For testing, enter:

H/5 * * * *

This means Jenkins schedules the job approximately every **5 minutes**, using a hashed minute to help distribute load.

Alternatively, for simple documentation, you can use:

*/5 * * * *



Step 8: Understand Build Periodically

Unlike Poll SCM, **Build Periodically does not check whether Git has changed.**

It simply runs the Jenkins job according to the schedule.

For example:

Every 5 minutes



Jenkins starts the job



Build executes

Even if there are no new Git changes, the scheduled build can still run.

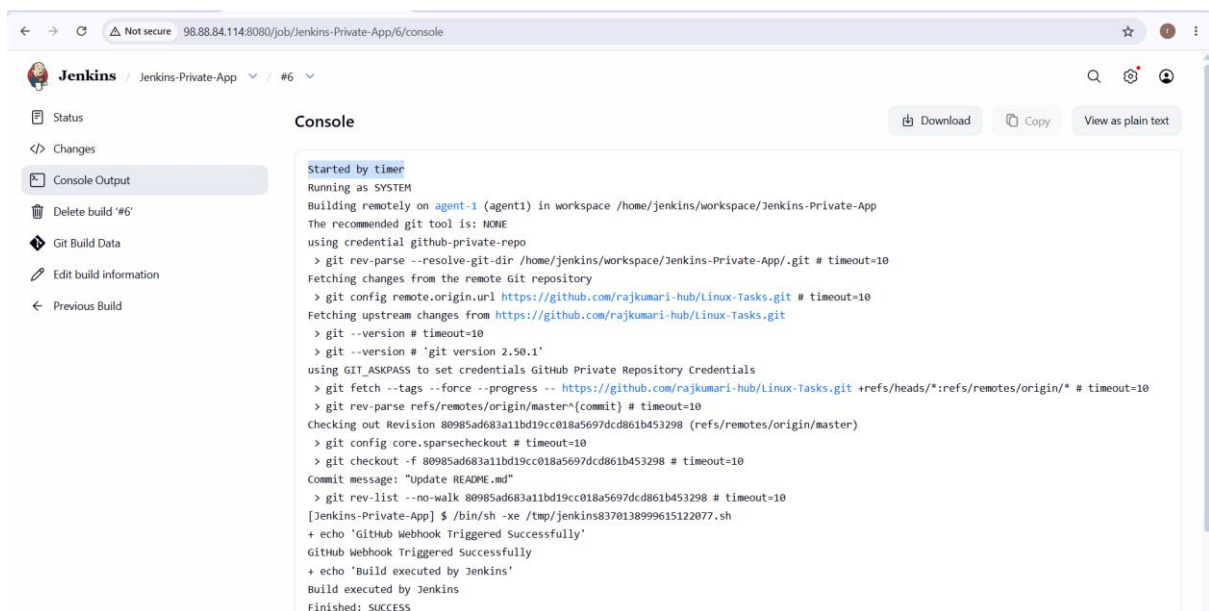
Part C: Save the Configuration

Step 9: Save the Job

Scroll to the bottom and click:

Save

Your job now has both triggers configured.



The screenshot shows the Jenkins web interface for a job named 'Jenkins-Private-App'. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Delete build '#6', Git Build Data, Edit build information, and Previous Build. The main area displays the 'Console' output, which shows the build process triggered by a timer. The output includes commands for setting up the git environment, fetching changes from a remote repository, and checking out the latest revision. The build concludes with a successful status.

```
Started by timer
Running as SYSTEM
Building remotely on agent-1 (agent1) in workspace /home/jenkins/workspace/Jenkins-Private-App
The recommended git tool is: NONE
using credential github-private-repo
> git rev-parse --resolve-git-dir /home/jenkins/workspace/Jenkins-Private-App/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/rajkumari-hub/Linux-Tasks.git # timeout=10
Fetching upstream changes from https://github.com/rajkumari-hub/Linux-Tasks.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials GitHub Private Repository Credentials
> git fetch --tags --force --progress -- https://github.com/rajkumari-hub/Linux-Tasks.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision 80985ad683a11bd19cc018a5697dcd861b453298 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f 80985ad683a11bd19cc018a5697dcd861b453298 # timeout=10
Commit message: "Update README.md"
> git rev-list --no-walk 80985ad683a11bd19cc018a5697dcd861b453298 # timeout=10
[Jenkins-Private-App] $ /bin/sh -xe /tmp/jenkins8370138999615122077.sh
+ echo 'GitHub Webhook Triggered Successfully'
GitHub Webhook Triggered Successfully
+ echo 'Build executed by Jenkins'
Build executed by Jenkins
Finished: SUCCESS
```

Step 10: Verify Poll SCM

Go to:

Jenkins → Jenkins-Private-App → Configure

Confirm:

Build Triggers

☒ Poll SCM

Schedule:

`*/5 * * * *`

Step 11: Verify Build Periodically

Confirm:

☒ Build periodically

Schedule:

`H/5 * * * *`

Difference Between Poll SCM and Build Periodically

Feature	Poll SCM	Build Periodically
Checks Git repository	☒ Yes	☒ No
Requires source-code change to trigger build	☒ Yes	☒ No
Runs according to schedule	☒ Checks according to schedule	☒ Builds according to schedule
Purpose	Detect source-code changes	Run jobs at fixed intervals
Example	<code>*/5 * * * *</code>	<code>H/5 * * * *</code>

This is Poll SCM Console OUTPUT

This is Build Periodically Console OUTPUT

4. Take a backup of the Jenkins server using a bash-script.

Objective: To create a **Bash script** that takes a backup of the Jenkins server by compressing the Jenkins home directory into a .tar.gz file.

Prerequisites

Before starting, make sure:

- Jenkins is installed and running.
- You have sudo privileges.
- Jenkins home directory is available.
- Sufficient disk space is available for the backup.

Check Jenkins status:

Verify its contents:

```
sudo ls -l /var/lib/jenkins
```

You may see directories such as:

jobs

plugins

users

workspace

secrets

```
ec2-user@ip-172-31-19-206:~
[ec2-user@ip-172-31-19-206 ~]$ echo $JENKINS_HOME
[ec2-user@ip-172-31-19-206 ~]$ sudo systemctl cat jenkins | grep JENKINS_HOME
# $JENKINS_HOME, $JENKINS_LOG, and (if you have already run Jenkins)
Environment="$JENKINS_HOME=/var/lib/jenkins"
# Note also that the particularly sensitive parts of $JENKINS_HOME (such as
[ec2-user@ip-172-31-19-206 ~]$ sudo ls -ld /var/lib/jenkins
drwxr-xr-x. 17 jenkins jenkins 16384 Aug 12 14:18 /var/lib/jenkins
[ec2-user@ip-172-31-19-206 ~]$ sudo ls -l /var/lib/jenkins
total 128
-rw-r--r--. 1 jenkins jenkins 2729 Aug 12 08:44 config.xml
-rw-r--r--. 1 jenkins jenkins 2443 Aug 12 11:36 credentials.xml
drwxr-xr-x. 7 jenkins jenkins 56 Aug 12 11:20 fingerprints
-rw-r--r--. 1 jenkins jenkins 156 Aug 12 08:44 hudson.model.UpdateCenter.xml
-rw-r--r--. 1 jenkins jenkins 370 Aug 11 07:50 hudson.plugins.git.GitTool.xml
-rw-r--r--. 1 jenkins jenkins 1680 Aug 11 07:50 identity.key.enc
-rw-r--r--. 1 jenkins jenkins 307 Aug 11 09:22 io.jenkins.plugins.prism.PrismConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 7 Aug 12 08:44 jenkins.install.InstallUtil.lastExecVersion
-rw-r--r--. 1 jenkins jenkins 7 Aug 11 08:13 jenkins.install.UpgradeWizard.state
-rw-r--r--. 1 jenkins jenkins 181 Aug 11 08:13 jenkins.model.JenkinsLocationConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 241 Aug 11 09:22 jenkins.plugins.git.GitHooksConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 169 Aug 11 09:22 jenkins.security.QueueItemAuthenticatorConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 162 Aug 11 09:22 jenkins.security.UpdateSiteWarningsConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 357 Aug 11 09:22 jenkins.security.apitoken.ApiTokenPropertyConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 171 Aug 11 07:12 jenkins.telemetry.Correlator.xml
drwxr-xr-x. 5 jenkins jenkins 73 Aug 12 11:54 jobs
drwxr-xr-x. 4 jenkins jenkins 123 Aug 12 12:28 logs
-rw-r--r--. 1 jenkins jenkins 1043 Aug 12 08:44 nodeMonitors.xml
drwxr-xr-x. 4 jenkins jenkins 36 Aug 12 11:35 nodes
-rw-r--r--. 1 jenkins jenkins 325 Aug 11 09:22 org.jenkinsci.plugins.gitclient.GitHostKeyVerificationConfiguration.xml
drwxr-xr-x. 94 jenkins jenkins 16384 Aug 11 07:50 plugins
-rw-r--r--. 1 jenkins jenkins 259 Aug 12 14:18 queue.xml
-rw-r--r--. 1 jenkins jenkins 258 Aug 12 08:44 queue.xml.bak
-rw-r--r--. 1 jenkins jenkins 334 Aug 11 09:22 scriptApproval.xml
-rw-r--r--. 1 jenkins jenkins 64 Aug 11 07:12 secret.key
-rw-r--r--. 1 jenkins jenkins 0 Aug 11 07:12 secret.key.not-so-secret
drwx----- 2 jenkins jenkins 16384 Aug 11 10:42 secrets
drwxr-xr-x. 2 jenkins jenkins 149 Aug 12 08:22 updates
drwxr-xr-x. 2 jenkins jenkins 24 Aug 11 07:12 userContent
drwxr-xr-x. 6 jenkins jenkins 16384 Aug 12 12:39 users
drwxr-xr-x. 6 jenkins jenkins 129 Aug 11 12:20 workspace
[ec2-user@ip-172-31-19-206 ~]$
```

4. Create a Backup Directory

Create a directory where the Jenkins backup will be stored:

```
sudo mkdir -p /backup/jenkins
```

Verify:

```
sudo ls -ld /backup/jenkins
```

Expected:

```
/backup/jenkins
```

```
ec2-user@ip-172-31-19-206:~  
[ec2-user@ip-172-31-19-206 ~]$ sudo mkdir -p /backup/jenkins  
[ec2-user@ip-172-31-19-206 ~]$ sudo ls -ld /backup/jenkins  
drwxr-xr-x. 2 root root 6 Aug 12 14:21 /backup/jenkins  
[ec2-user@ip-172-31-19-206 ~]$ |
```

5. Create the Bash Backup Script

Create a script:

```
sudo vi /usr/local/bin/jenkins_backup.sh
```

Add the following content:

```
ec2-user@ip-172-31-19-206:~  
#!/bin/bash  
  
# Jenkins Backup Script  
  
JENKINS_HOME="/var/lib/jenkins"  
BACKUP_DIR="/backup/jenkins"  
DATE=$(date +%Y-%m-%d_%H-%M-%S)  
BACKUP_FILE="$BACKUP_DIR/jenkins_backup_$DATE.tar.gz"  
  
echo "=====  
echo "      Jenkins Backup Started"  
echo "=====  
  
echo "Jenkins Home : $JENKINS_HOME"  
echo "Backup File  : $BACKUP_FILE"  
  
# Create backup directory  
mkdir -p "$BACKUP_DIR"  
  
# Create compressed backup  
tar -czpf "$BACKUP_FILE" "$JENKINS_HOME"  
  
# Check backup status  
if [ $? -eq 0 ]; then  
    echo "Jenkins backup completed successfully."  
    echo "Backup location: $BACKUP_FILE"  
else  
    echo "Jenkins backup failed."  
    exit 1  
fi  
  
echo "=====  
echo "      Jenkins Backup Completed"  
echo "=====  
~  
~
```

6. Give Execute Permission to the Script

Run:

```
sudo chmod +x /usr/local/bin/jenkins_backup.sh
```

Verify:

```
ls -l /usr/local/bin/jenkins_backup.sh
```

You should see execute permission:

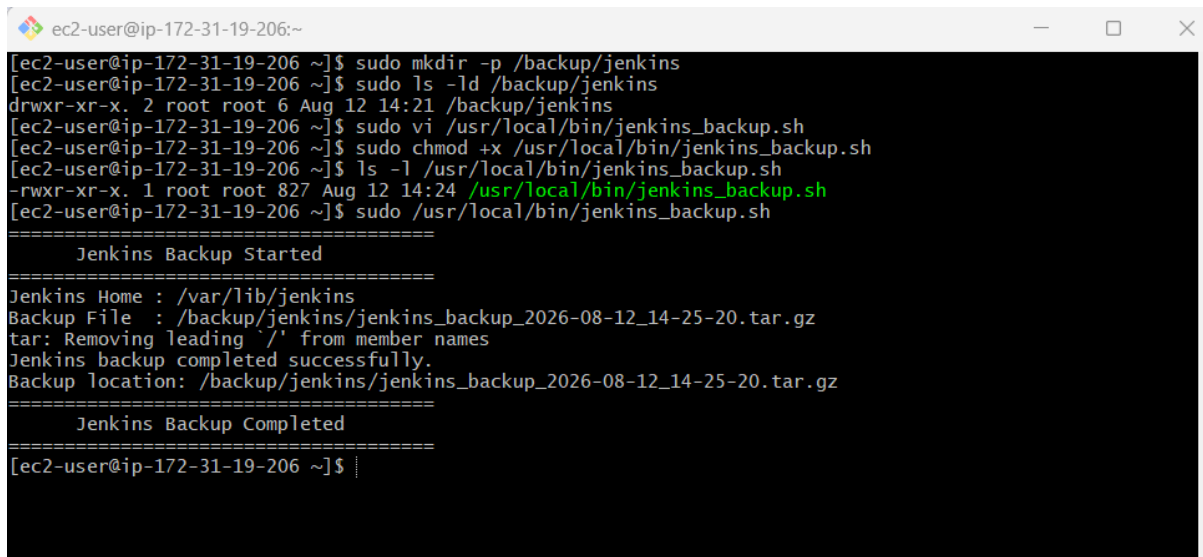
```
-rwxr-xr-x
```

7. Execute the Backup Script

Run:

```
sudo /usr/local/bin/jenkins_backup.sh
```

Example output:

A terminal window titled 'ec2-user@ip-172-31-19-206:~' showing the execution of the Jenkins backup script. The user runs 'sudo mkdir -p /backup/jenkins', 'sudo ls -ld /backup/jenkins' (showing permissions drwxr-xr-x), 'sudo vi /usr/local/bin/jenkins_backup.sh', 'sudo chmod +x /usr/local/bin/jenkins_backup.sh', 'ls -l /usr/local/bin/jenkins_backup.sh' (showing permissions -rwxr-xr-x), and finally 'sudo /usr/local/bin/jenkins_backup.sh'. The script output includes 'Jenkins Backup Started', 'Jenkins Home : /var/lib/jenkins', 'Backup File : /backup/jenkins/jenkins_backup_2026-08-12_14-25-20.tar.gz', 'tar: Removing leading '/' from member names', 'Jenkins backup completed successfully.', 'Backup location: /backup/jenkins/jenkins_backup_2026-08-12_14-25-20.tar.gz', and 'Jenkins Backup Completed'. The prompt returns to '[ec2-user@ip-172-31-19-206 ~]\$'.

8. Verify the Backup File

List the backup directory:

```
sudo ls -lh /backup/jenkins/
```

9. Verify the Backup Contents

To check what was backed up: `sudo tar -tzf /backup/jenkins/jenkins_backup_2026-08-12_19-50-30.tar.gz | head -20`

10. Check the Backup File Size

Run:

```
sudo du -sh /backup/jenkins/
```

Example:

```
25M /backup/jenkins
```

```
[ec2-user@ip-172-31-19-206 ~]$ sudo tar -tzf /backup/jenkins/jenkins_backup_2026-08-12_14-25-20.tar.gz | h
ead -20
var/lib/jenkins/
var/lib/jenkins/.java/
var/lib/jenkins/.java/fonts/
var/lib/jenkins/.java/fonts/21.0.12/
var/lib/jenkins/.java/fonts/21.0.12/fcinfo-1-ip-172-31-19-206.ec2.internal-amzn-2023-en-.properties
var/lib/jenkins/secret.key
var/lib/jenkins/secret.key.not-so-secret
var/lib/jenkins/plugins/
var/lib/jenkins/plugins/commons-lang3-api.jpi
var/lib/jenkins/plugins/commons-lang3-api/
var/lib/jenkins/plugins/commons-lang3-api/META-INF/
var/lib/jenkins/plugins/commons-lang3-api/META-INF/MANIFEST.MF
var/lib/jenkins/plugins/commons-lang3-api/META-INF/maven/
var/lib/jenkins/plugins/commons-lang3-api/META-INF/maven/io.jenkins.plugins/
var/lib/jenkins/plugins/commons-lang3-api/META-INF/maven/io.jenkins.plugins/commons-lang3-api/
var/lib/jenkins/plugins/commons-lang3-api/META-INF/maven/io.jenkins.plugins/commons-lang3-api/pom.xml
var/lib/jenkins/plugins/commons-lang3-api/META-INF/maven/io.jenkins.plugins/commons-lang3-api/pom.properties
var/lib/jenkins/plugins/commons-lang3-api/WEB-INF/
var/lib/jenkins/plugins/commons-lang3-api/WEB-INF/lib/
var/lib/jenkins/plugins/commons-lang3-api/WEB-INF/lib/commons-lang3-3.20.0.jar
[ec2-user@ip-172-31-19-206 ~]$ sudo du -sh /backup/jenkins/
271M /backup/jenkins/
[ec2-user@ip-172-31-19-206 ~]$ |
```

5. Take a backup of Jenkins using the **Thin Backup** plugin.

Objective: To install and configure the **ThinBackup plugin** in Jenkins and create a backup of Jenkins configuration and important Jenkins data.

2. Prerequisites

Before starting:

- Jenkins should be installed and running.
- You should have Jenkins administrator access.
- You should have sufficient disk space for the backup.
- A separate backup directory should preferably be used.

Check Jenkins:

```
sudo systemctl status jenkins
```

Expected:

Active: active (running)

3. Open Jenkins Dashboard

Open Jenkins in your browser:

http://<JENKINS-IP>:8080

Log in using your Jenkins administrator account.

4. Open Plugin Management

From the Jenkins dashboard:

Jenkins Dashboard



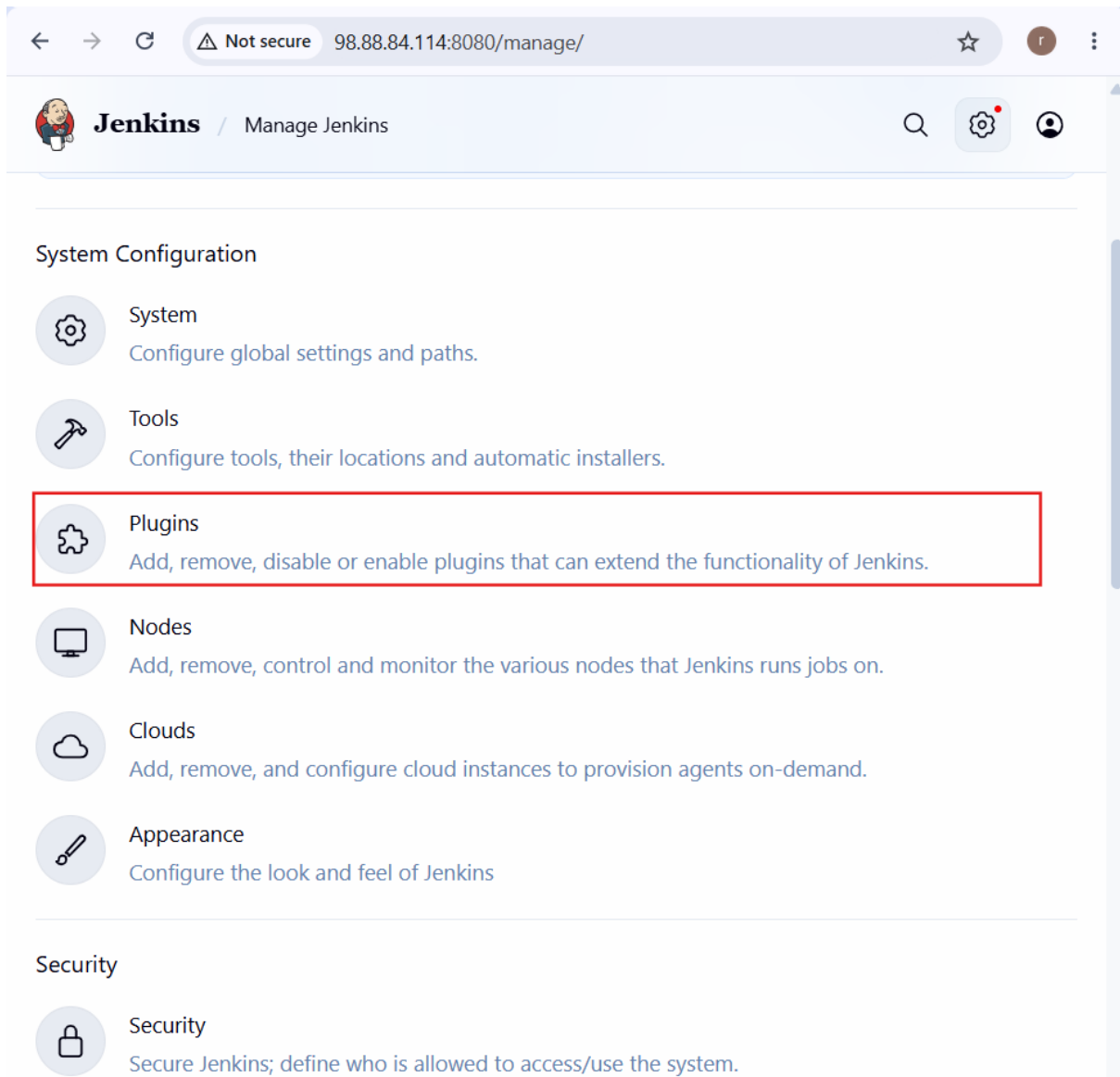
Manage Jenkins



Plugins

Depending on your Jenkins version, you may see:

Manage Jenkins → Plugins



5. Search for ThinBackup

Go to the **Available plugins** section.

Search for:

ThinBackup

Select:

ThinBackup

Then click:

Install

If Jenkins asks you to restart, restart Jenkins after the installation completes.



Jenkins

/ Manage Jenkins



/ Plugins



Plugins

📄 Updates

📦 Available plugins

⚙ Installed plugins

⚙ Advanced settings

🔍 ThinBackup

📄 Install



Install

Name ↓

Released

Health



ThinBackup 2.1.4

Miscellaneous

Backups the most important global and job specific configuration files.

4 mo 3 days ago

100

Plugins

📄 Updates

📦 Available plugins

⚙️ Installed plugins

⚙️ Advanced settings

☰ Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

ThinBackup ✓ Success

Loading plugin extensions ✓ Success

→ [Go back to the top page](#)
(you can start using the installed plugins right away)

6. Verify the Plugin Installation

Go to:

Manage Jenkins

Search for **ThinBackup**.

You should now have a ThinBackup configuration option.

Depending on the Jenkins version/plugin UI, it may appear under:

Manage Jenkins → ThinBackup

or through the plugin's configuration page.

7. Create a Backup Directory

It is better to create a dedicated directory for ThinBackup.

On your EC2 Jenkins server:

```
sudo mkdir -p /backup/thinbackup
```

Give Jenkins permission to use it:

```
sudo chown -R jenkins:jenkins /backup/thinbackup
```

Verify:

```
sudo ls -ld /backup/thinbackup
```

You should see the Jenkins user/group as the owner.

```
[ec2-user@ip-172-31-19-206 ~]$ sudo mkdir -p /backup/thinbackup
[ec2-user@ip-172-31-19-206 ~]$ sudo su -
Last login: Wed Aug 12 11:58:51 UTC 2026 on pts/7
[root@ip-172-31-19-206 ~]# sudo mkdir -p /backup/thinbackup
[root@ip-172-31-19-206 ~]# sudo chown -R jenkins:jenkins /backup/thinbackup
[root@ip-172-31-19-206 ~]# ll
total 0
[root@ip-172-31-19-206 ~]# sudo ls -ld /backup/thinbackup
drwxr-xr-x. 2 jenkins jenkins 6 Aug 12 14:43 /backup/thinbackup
[root@ip-172-31-19-206 ~]#
```

 **Jenkins** / Manage Jenkins



[See the version and license information.](#)

Troubleshooting

 **Manage Old Data**
Scrub configuration files to remove remnants from old plugins and earlier versions.

Tools and Actions

 **Reload Configuration from Disk**
Discard all the loaded data in memory and reload everything from file system. Useful when you modified config files directly on disk.

 **Jenkins CLI**
Access/manage Jenkins from your shell, or from your script.

 **Script Console**
Executes arbitrary script for administration/trouble-shooting/diagnostics.

 **ThinBackup**
Backup your global and job specific configuration.

 **Prepare for Shutdown**
Stops executing new builds, so that the system can be eventually shut down safely.

Step 7: Configure the Backup Directory

On the ThinBackup page, find:

Backup directory

Enter:

```
/backup/thinbackup
```

This tells ThinBackup where to store the backups.

The Jenkins process needs write access to this directory, which you have already configured.



Jenkins

Manage Jenkins

System



Global Untrusted Pipeline Libraries

Sharable libraries available to any Pipeline jobs running on this system. These libraries will be untrusted, meaning they run with "sandbox" restrictions and cannot use @Grab.

+ Add

ThinBackup Configuration

Backup directory ?

/backup/thinbackup

Backup schedule for full backups ?

Backup schedule for differential backups ?

Max number of backup sets ?

-1

Save

Apply

```
aws
Search [Alt+S]
United States (N. Virginia) Rajkumari (57183338)

/m/
Last login: Wed Aug 12 14:16:52 2026 from 103.248.210.235
[ec2-user@ip-172-31-19-206 ~]$ sudo su -
Last login: Wed Aug 12 14:43:48 UTC 2026 on pts/1
[root@ip-172-31-19-206 ~]# sudo ls -lh /backup/thinbackup/
total 32K
drwxr-xr-x. 5 jenkins jenkins 16K Aug 12 15:10 FULL-2026-08-12_15-10
drwxr-xr-x. 5 jenkins jenkins 16K Aug 12 15:13 FULL-2026-08-12_15-13
[root@ip-172-31-19-206 ~]# sudo du -sh /backup/thinbackup/
816K /backup/thinbackup/
[root@ip-172-31-19-206 ~]# sudo find /backup/thinbackup/FULL-2026-08-12_15-13 -maxdepth 3 -type f | head -30
sudo find /backup/thinbackup/FULL-2026-08-12_15-13 -m
h 3 -type f | head -30
/backup/thinbackup/FULL-2026-08-12_15-13/jobs/Test-agent-1/config.xml
/backup/thinbackup/FULL-2026-08-12_15-13/jobs/Test-agent-2/config.xml
/backup/thinbackup/FULL-2026-08-12_15-13/jobs/Jenkins-Private-App/config.xml
/backup/thinbackup/FULL-2026-08-12_15-13/users/Rajkumari.00e8d7223168b2ab478d78f2eb4c0b9446fdb4705bfc41906a34399a250f8c12/config.xml
/backup/thinbackup/FULL-2026-08-12_15-13/users/devops.7b90c5e4bb3bffd2bd7fe227d52789a4407947c28e4ab00419686568244ac4e/config.xml
/backup/thinbackup/FULL-2026-08-12_15-13/users/testing.7e853fafa7dfcccd1719813141ed5fcd4fad0ab664f5405108b1596dae894a5e1/config.xml
/backup/thinbackup/FULL-2026-08-12_15-13/users/noreply.724f299bd31dc57a7dd3e6a80522d2914d9434aaa85fc873262265b9d369f08a/config.xml
/backup/thinbackup/FULL-2026-08-12_15-13/nodes/agent-1/config.xml
/backup/thinbackup/FULL-2026-08-12_15-13/nodes/agent-2/config.xml
/backup/thinbackup/FULL-2026-08-12_15-13/installedPlugins.xml
/backup/thinbackup/FULL-2026-08-12_15-13/backup-completed.info
/backup/thinbackup/FULL-2026-08-12_15-13/hudson.model.UpdateCenter.xml
/backup/thinbackup/FULL-2026-08-12_15-13/jenkins.telemetry.Correlator.xml
/backup/thinbackup/FULL-2026-08-12_15-13/nodeMonitors.xml
/backup/thinbackup/FULL-2026-08-12_15-13/config.xml
/backup/thinbackup/FULL-2026-08-12_15-13/hudson.plugins.git.GitTool.xml
/backup/thinbackup/FULL-2026-08-12_15-13/jenkins.model.JenkinsLocationConfiguration.xml
/backup/thinbackup/FULL-2026-08-12_15-13/io.jenkins.plugins.prism.PrismConfiguration.xml
/backup/thinbackup/FULL-2026-08-12_15-13/jenkins.plugins.git.GitHooksConfiguration.xml
/backup/thinbackup/FULL-2026-08-12_15-13/jenkins.security.QueueItemAuthenticatorConfiguration.xml
```

6. Set up a new Jenkins server and restore the backup taken in task 4.

Step 1 — On the OLD Jenkins server

Verify your Task 4 backup: `sudo ls -lh /backup/jenkins/`

You should see:

`jenkins_backup_2026-08-12_14-25-20.tar.gz`

Step 2 — Create the NEW EC2 Jenkins server

Launch a new EC2 instance with:

- Amazon Linux 2023
- Java
- Jenkins
- Port 22 for SSH
- Port 8080 for Jenkins

Install and start Jenkins on the new server.

```
root@ip-172-31-5-154:~
[root@ip-172-31-5-154 ~]# sudo dnf install java-21-amazon-corretto -y
Last metadata expiration check: 0:04:09 ago on Wed Aug 12 15:50:14 2026.
Package java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-5-154 ~]# java --version
openjdk 21.0.12 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
[root@ip-172-31-5-154 ~]# sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/rpm-stable/jenkins.repo
--2026-08-12 15:55:28-- https://pkg.jenkins.io/rpm-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.38.133, 2a04:4e42:77::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|146.75.38.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 267 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jenkins.r 100%[=====>] 267 --.-KB/s in 0s

2026-08-12 15:55:28 (10.4 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [267/267]

[root@ip-172-31-5-154 ~]# sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
[root@ip-172-31-5-154 ~]# sudo dnf upgrade -y
Jenkins-stable                               12 kB/s | 833 B      00:00
Jenkins-stable                               101 kB/s | 1.6 kB    00:00
Importing GPG key 0x14ABFC68:
  Userid : "Jenkins Project <jenkinsci-board@googlegroups.com>"
  Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
  From : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Jenkins-stable                               35 kB/s | 2.6 kB     00:00
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-5-154 ~]#
```

```

root@ip-172-31-5-154:~
[root@ip-172-31-5-154 ~]# sudo dnf install jenkins -y
Last metadata expiration check: 0:00:45 ago on Wed Aug 12 15:55:52 2026.
Dependencies resolved.
=====
Package                Architecture    Version      Repository    Size
=====
Installing:
jenkins                noarch         2.568.2-1    jenkins       96 M
=====
Transaction Summary
=====
Install 1 Package

Total download size: 96 M
Installed size: 96 M
Downloading Packages:
jenkins-2.568.2-1.noarch.rpm                34 MB/s | 96 MB    00:02
-----
Total                                         34 MB/s | 96 MB    00:02
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing                :                               1/1
  Running scriptlet: jenkins-2.568.2-1.noarch             1/1
  Installing           : jenkins-2.568.2-1.noarch         1/1
  Running scriptlet: jenkins-2.568.2-1.noarch             1/1
  Verifying            : jenkins-2.568.2-1.noarch         1/1

Installed:
jenkins-2.568.2-1.noarch

Complete!
[root@ip-172-31-5-154 ~]# sudo systemctl status jenkins
Unit jenkins.service could not be found.
Unit systemctl.service could not be found.
Unit status.service could not be found.
o jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)
   Active: inactive (dead)
[root@ip-172-31-5-154 ~]# sudo systemctl status jenkins
o jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)
   Active: inactive (dead)
[root@ip-172-31-5-154 ~]# sudo systemctl start jenkins
[root@ip-172-31-5-154 ~]# sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)
   Active: active (running) since Wed 2026-08-12 15:58:02 UTC; 9s ago
     Main PID: 26494 (java)
       Tasks: 43 (limit: 1059)
      Memory: 388.7M
         CPU: 19.857s
    CGroup: /system.slice/jenkins.service
            └─26494 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var

Aug 12 15:57:57 ip-172-31-5-154.ec2.internal jenkins[26494]: [LF]>
Aug 12 15:57:57 ip-172-31-5-154.ec2.internal jenkins[26494]: [LF]> *****
Aug 12 15:57:57 ip-172-31-5-154.ec2.internal jenkins[26494]: [LF]> *****
Aug 12 15:57:57 ip-172-31-5-154.ec2.internal jenkins[26494]: [LF]> *****
Aug 12 15:58:02 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 15:58:02.963+0000 [id=32]
Aug 12 15:58:02 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 15:58:02.981+0000 [id=24]
Aug 12 15:58:02 ip-172-31-5-154.ec2.internal systemd[1]: Started jenkins.service - Jenkins Continuous Int
Aug 12 15:58:03 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 15:58:03.088+0000 [id=50]
Aug 12 15:58:03 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 15:58:03.091+0000 [id=50]
Aug 12 15:58:08 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 15:58:08.049+0000 [id=70]

[root@ip-172-31-5-154 ~]#

```

Step 3 — Stop Jenkins on the NEW server

Before restoring /var/lib/jenkins, stop Jenkins:

```
sudo systemctl stop jenkins
```

Verify:

```
sudo systemctl status jenkins
```

It should be stopped.


```
root@ip-172-31-5-154:~  
[root@ip-172-31-5-154 ~]# sudo systemctl stop jenkins  
[root@ip-172-31-5-154 ~]# sudo systemctl status jenkins  
o jenkins.service - Jenkins Continuous Integration Server  
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)  
   Active: inactive (dead)  
  
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.867+0000 [id=26]  
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.870+0000 [id=26]  
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.893+0000 [id=26]  
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.904+0000 [id=26]  
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.915+0000 [id=26]  
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.916+0000 [id=26]  
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.924+0000 [id=26]  
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal systemd[1]: jenkins.service: Deactivated successfully.  
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal systemd[1]: Stopped jenkins.service - Jenkins Continuous Int  
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal systemd[1]: jenkins.service: Consumed 20.287s CPU time.  
  
[root@ip-172-31-5-154 ~]# AC  
[root@ip-172-31-5-154 ~]# AC  
[root@ip-172-31-5-154 ~]# |
```

Step 4 — Copy the backup from OLD to NEW server

For example, from the old server:

```
scp -i your-key.pem \  
/backup/jenkins/jenkins_backup_2026-08-12_14-25-20.tar.gz \  
ec2-user@<NEW-SERVER-IP>:/tmp/
```

Then on the new server:

```
ls -lh /tmp/jenkins_backup_2026-08-12_14-25-20.tar.gz
```

You should see approximately:

271M

Step 6 — Extract your Task 4 backup

Your backup was created from the Jenkins home directory, so restore it with:

```
sudo tar -xzipf /tmp/jenkins_backup_2026-08-12_14-25-20.tar.gz -C /
```

This should recreate:

```
/var/lib/jenkins
```

Step 7 — Verify the restored files

Run:

```
sudo ls -lah /var/lib/jenkins
```

You should see files/directories such as:

```
config.xml
```

```
jobs
```

```
users
```

```
plugins
```

```
workspace
```

```
secrets
```

```
nodes
```

You can also check the size:

```

ec2-user@ip-172-31-5-154:~
[ec2-user@ip-172-31-5-154 ~]$ sudo systemctl stop jenkins
[ec2-user@ip-172-31-5-154 ~]$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)
   Active: inactive (dead)

Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.867+0000 [id=
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.870+0000 [id=
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.893+0000 [id=
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.904+0000 [id=
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.913+0000 [id=
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.916+0000 [id=
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal jenkins[26494]: 2026-08-12 16:00:12.924+0000 [id=
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal systemd[1]: jenkins.service: Deactivated successf
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal systemd[1]: Stopped jenkins.service - Jenkins Con
Aug 12 16:00:12 ip-172-31-5-154.ec2.internal systemd[1]: jenkins.service: Consumed 20.287s CPU

[ec2-user@ip-172-31-5-154 ~]$ sudo mv /var/lib/jenkins /var/lib/jenkins-before-restore
[ec2-user@ip-172-31-5-154 ~]$ sudo ls -ld /var/lib/jenkins-before-restore
drwxr-xr-x. 10 jenkins jenkins 16384 Aug 12 16:00 /var/lib/jenkins-before-restore
[ec2-user@ip-172-31-5-154 ~]$ sudo tar -xpf /tmp/jenkins_backup_2026-08-12_14-25-20.tar.gz -C
[ec2-user@ip-172-31-5-154 ~]$ sudo tar -xpf /tmp/jenkins_backup_2026-08-12_14-25-20.tar.gz -C
[ec2-user@ip-172-31-5-154 ~]$ sudo ls -lah /var/lib/jenkins
total 164K
drwxr-xr-x. 17 jenkins jenkins 16K Aug 12 14:23 .
drwxr-xr-x. 28 root root 16K Aug 12 16:35 ..
drwxr-xr-x. 3 jenkins jenkins 17 Aug 11 07:12 .cache
drwxr-xr-x. 3 jenkins jenkins 18 Aug 11 10:57 .config
drwxr-xr-x. 3 jenkins jenkins 20 Aug 11 07:50 .groovy
drwxr-xr-x. 3 jenkins jenkins 19 Aug 11 07:12 .java
-rw-r--r--. 1 jenkins jenkins 0 Aug 12 08:44 .lastStarted
-rw-r--r--. 1 jenkins jenkins 4 Aug 12 14:10 .owner
drwxr-xr-x. 2 jenkins jenkins 88 Aug 12 10:12 .ssh
-rw-r--r--. 1 jenkins jenkins 2.7K Aug 12 08:44 config.xml
-rw-r--r--. 1 jenkins jenkins 2.4K Aug 12 11:36 credentials.xml
drwxr-xr-x. 7 jenkins jenkins 56 Aug 12 11:20 fingerprint
-rw-r--r--. 1 jenkins jenkins 156 Aug 12 08:44 hudson.model.UpdateCenter.xml
-rw-r--r--. 1 jenkins jenkins 370 Aug 11 07:50 hudson.plugins.git.GitTool.xml
-rw-r--r--. 1 jenkins jenkins 1.7K Aug 11 07:50 identity.key.enc
-rw-r--r--. 1 jenkins jenkins 307 Aug 11 09:22 io.jenkins.plugins.prisa.PrisaConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 7 Aug 12 08:44 jenkins.install.InstallUtils.lastExecVersion
-rw-r--r--. 1 jenkins jenkins 7 Aug 11 08:13 jenkins.install.UpgradeWizard.state
-rw-r--r--. 1 jenkins jenkins 181 Aug 11 08:13 jenkins.model.JenkinsLocationConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 241 Aug 11 09:22 jenkins.plugins.git.GitHooksConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 169 Aug 11 09:22 jenkins.security.QueueItemAuthenticatorConfigu
ration.xml
-rw-r--r--. 1 jenkins jenkins 162 Aug 11 09:22 jenkins.security.UpdateSiteWarningsConfigurati
on.xml
-rw-r--r--. 1 jenkins jenkins 357 Aug 11 09:22 jenkins.security.ApiToken.ApiTokenPropertyConf
iguration.xml
-rw-r--r--. 1 jenkins jenkins 171 Aug 11 07:12 jenkins.telemetry.Correlator.xml
drwxr-xr-x. 5 jenkins jenkins 73 Aug 12 11:54 jobs
drwxr-xr-x. 4 jenkins jenkins 123 Aug 12 12:28 logs
-rw-r--r--. 1 jenkins jenkins 1.1K Aug 12 08:44 nodeMonitors.xml
drwxr-xr-x. 4 jenkins jenkins 36 Aug 12 11:35 nodes
-rw-r--r--. 1 jenkins jenkins 325 Aug 11 09:22 org.jenkinsci.plugins.gitclient.GitHostKeyVeri
ficationConfiguration.xml
drwxr-xr-x. 94 jenkins jenkins 16K Aug 11 07:50 plugins
-rw-r--r--. 1 jenkins jenkins 259 Aug 12 14:23 queue.xml
-rw-r--r--. 1 jenkins jenkins 258 Aug 12 08:44 queue.xml.bak
-rw-r--r--. 1 jenkins jenkins 334 Aug 11 09:22 scriptApproval.xml
-rw-r--r--. 1 jenkins jenkins 64 Aug 11 07:12 secret.key
-rw-r--r--. 1 jenkins jenkins 0 Aug 11 07:12 secret.key.not-so-secret
drwxr-xr-x. 2 jenkins jenkins 16K Aug 11 10:42 secrets
drwxr-xr-x. 2 jenkins jenkins 149 Aug 12 08:22 updates
drwxr-xr-x. 2 jenkins jenkins 24 Aug 11 07:12 userContent
drwxr-xr-x. 6 jenkins jenkins 16K Aug 12 12:19 users

```

You can also check the size:

```
sudo du -sh /var/lib/jenkins
```

Step 8 — Fix Jenkins ownership

Very important:

```
sudo chown -R jenkins:jenkins /var/lib/jenkins
```

Then:

```
sudo ls -ld /var/lib/jenkins
```

Expected:

```
jenkins jenkins
```

Step 9 — Start Jenkins

```
sudo systemctl start jenkins
```

Check:

```
sudo systemctl status jenkins
```

You want:

Active: active (running)

```
ec2-user@ip-172-31-5-154:~  
[ec2-user@ip-172-31-5-154 ~]$ sudo du -sh /var/lib/jenkins  
323M    /var/lib/jenkins  
[ec2-user@ip-172-31-5-154 ~]$ sudo chown -R jenkins:jenkins /var/lib/jenkins  
[ec2-user@ip-172-31-5-154 ~]$ sudo ls -ld /var/lib/jenkins  
drwxr-xr-x. 17 jenkins jenkins 16384 Aug 12 14:23 /var/lib/jenkins  
[ec2-user@ip-172-31-5-154 ~]$ sudo systemctl start jenkins  
[ec2-user@ip-172-31-5-154 ~]$ sudo systemctl status jenkins  
● jenkins.service - Jenkins Continuous Integration Server  
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)  
   Active: active (running) since Wed 2026-08-12 16:40:06 UTC; 22s ago  
     Main PID: 28461 (java)  
       Tasks: 58 (limit: 1059)  
      Memory: 332.8M  
         CPU: 26.551s  
    CGroup: /system.slice/jenkins.service  
            └─28461 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war ->  
  
Aug 12 16:40:05 ip-172-31-5-154.ec2.internal jenkins[28461]: 2026-08-12 16:40:05.551+0000 [id=>  
Aug 12 16:40:05 ip-172-31-5-154.ec2.internal jenkins[28461]: 2026-08-12 16:40:05.553+0000 [id=>  
Aug 12 16:40:05 ip-172-31-5-154.ec2.internal jenkins[28461]: 2026-08-12 16:40:05.556+0000 [id=>  
Aug 12 16:40:05 ip-172-31-5-154.ec2.internal jenkins[28461]: 2026-08-12 16:40:05.557+0000 [id=>  
Aug 12 16:40:05 ip-172-31-5-154.ec2.internal jenkins[28461]: 2026-08-12 16:40:05.763+0000 [id=>  
Aug 12 16:40:05 ip-172-31-5-154.ec2.internal jenkins[28461]: 2026-08-12 16:40:05.766+0000 [id=>  
Aug 12 16:40:05 ip-172-31-5-154.ec2.internal jenkins[28461]: 2026-08-12 16:40:05.908+0000 [id=>  
Aug 12 16:40:06 ip-172-31-5-154.ec2.internal jenkins[28461]: 2026-08-12 16:40:06.029+0000 [id=>  
Aug 12 16:40:06 ip-172-31-5-154.ec2.internal systemd[1]: Started jenkins.service - Jenkins Con>  
Aug 12 16:40:13 ip-172-31-5-154.ec2.internal jenkins[28461]: 2026-08-12 16:40:13.025+0000 [id=>  
  
[ec2-user@ip-172-31-5-154 ~]$ |
```

Step 10 — Open Jenkins GUI

Open:

<http://3.238.7.134:8080>

You should now see your **restored Jenkins environment**.

Step 11 — Verify your old jobs

Go to:

Jenkins Dashboard

Check whether your previous jobs are present.

For example, if your old server had:

Jenkins-Private-App

it should now appear on the new Jenkins server.

Also check:

- **Dashboard → Manage Jenkins**

- **Manage Jenkins → Nodes**
- **Manage Jenkins → Plugins**
- Job configurations
- Build history
- Pipeline configuration
- Users

This is my new-jenkins-server

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	Jenkins-Private-App	3 min 33 sec #12	4 hr 17 min #1	4.9 sec
✓	☀	Test-agent-1	5 hr 3 min #1	N/A	0.64 sec
✓	☀	Test-agent-2	4 hr 59 min #1	N/A	0.56 sec

This is my old-jenkins-server

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	Jenkins-Private-App	2 hr 16 min #13	4 hr 20 min #1	0.24 sec
✓	☀	Test-agent-1	5 hr 6 min #1	N/A	0.64 sec
✓	☀	Test-agent-2	5 hr 2 min #1	N/A	0.56 sec